

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

WEB · FIGMA · HTML · CSS · JS · СЕРВЕР

Интернет-магазин CORAL

Комплект методических указаний по этапам курса

Этапы 0–10

Назначение	сквозной том указаний от макета Figma до корзины и админки
Проект	Coral_Example
Состав	12 частей: этапы 0–10 и подробный разбор паттерна БЭМ
Год	2026

2026

Аннотация

Настоящий том объединяет методические указания учебного проекта CORAL. Части расположены в порядке курса: макет Figma, семантический HTML, CSS, БЭМ, модальные окна, JavaScript, серверный API, адаптив, объектно-ориентированный рефакторинг, база данных SQLite, корзина и админка. Отдельным дополнением к этапу 3 дан подробный разбор паттерна БЭМ.

Каждая часть сохраняет собственную внутреннюю структуру: аннотация, порядок работы, теория на примере CORAL, практикумы, типичные ошибки, контрольные вопросы и источники. Нумерация страниц тома сквозная.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

Том можно использовать как единое учебно-методическое издание: читать последовательно или открывать нужный этап по оглавлению.

Как работать с методическими указаниями

- Читайте части по порядку курса, если осваиваете проект с нуля.
- Если нужен один приём — переходите к соответствующей части по оглавлению тома.
- Практикумы выполняйте в каталоге указанного этапа репозитория Coral_Example.
- Подробный разбор БЭМ читайте после этапа 3 или вместо краткой части, если имена классов вызывают затруднение.

Оглавление

Часть 0. Макет Figma как техническое задание
Часть 1. Семантическая HTML-разметка
Часть 2. Визуальное оформление на CSS
Часть 3. Независимые блоки БЭМ
Часть 3+. Паттерн БЭМ подробно
Часть 4. Модальные окна входа и карточки товара
Часть 5. JavaScript: витрина, модальки, формы
Часть 6. Серверный API на трёх стеках
Часть 7. Адаптив и CSS по блокам
Часть 8. Рефакторинг JavaScript на классах
Часть 9. SQLite как хранилище
Часть 10. Корзина, заказ и админка

Состав тома

Часть	Этап	Файл курса
0. Макет Figma	stage-00	Coral.fig, design/tokens.md
1. HTML	stage-01-html	index.html, login.html, product.html
2. CSS	stage-02-css	токены, flex, grid
3. БЭМ	stage-03-bem	рефакторинг классов
3+. БЭМ подробно	дополнение	паттерн имён
4. Модалки	stage-04-modals	#modal-auth, #modal-product
5. JavaScript	stage-05-js	fetch, localStorage
6. Сервер	stage-06-server	PHP, Django, Node.js
7. Адаптив	stage-07-adaptive	1100 / 768 / 480
8. ООП	stage-08-oop	классы и модули
9. База данных	stage-09-database	SQLite
10. Магазин	stage-10-shop-admin	корзина и админка

КАК ПОЛЬЗОВАТЬСЯ ТОМОМ

Страницы нумеруются сквозь все части. Колонтитул тома: CORAL · Методические указания. Исходные отдельные файлы частей сохранены в архиве рядом с этим изданием.

Репозиторий

Учебный проект: https://github.com/kruzenshteyn/Coral_Example. Онлайн-макет: файл Coral в Figma курса. Дата комплекта: 15.09.2026.

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

FIGMA · МАКЕТ · ДИЗАЙН-СИСТЕМА

Макет Figma как техническое задание
многостраничного интернет-магазина

Методические указания на примере проекта CORAL

Этап 0 — Figma

Назначение	теоретическое изучение и практический разбор макета Coral.fig
Уровень	начальный / базовый
Исходный проект	Coral_Example / этап 0 — макет, design/, docs/course/00-figma.md
Формат работы	чтение файла Figma, съём токенов, карта блоков, подготовка к HTML

2026

Аннотация

Методические указания посвящены чтению макета Figma как технического задания на вёрстку многостраничного интернет-магазина. Материал построен вокруг открытого учебного проекта CORAL: файл Coral.fig, онлайн-копия макета, таблица токенов design/tokens.md и карта блоков docs/course/00-figma.md.

Издание объясняет устройство файла Figma, смысл фреймов и слоёв, работу инспектора свойств, сетку и шаг отступов, токены цвета и типографики, компоненты и Auto Layout, состояния интерфейса, экспорт ассетов и границу между тем, что уже задано макетом, и тем, что появится только в HTML, CSS и JavaScript. Практические фрагменты разбираются с точки зрения пользовательского сценария и передачи в код.

На этапе 0 код ещё не пишут. Задача — научиться снимать спецификацию: какие блоки есть на странице, какие значения повторяются, какие состояния скрыты на статичном кадре и какие решения нельзя «угадать по картинке».

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет открыть учебный макет CORAL, объяснить устройство файла Figma, снять сетку и токены, составить карту блоков с будущими HTML-тегами, отличить компонент от разовой картинки и найти типичные ошибки передачи макета в вёрстку.

Как работать с методическими указаниями

- Сначала откройте Coral.fig в Figma Desktop или по ссылке проекта; параллельно держите design/tokens.md и карту блоков этапа 0.
- Каждый пример сверяйте с реальным слоем: выделяйте блок, смотрите правую панель Inspect, записывайте числа, а не впечатление от скриншота.
- Ведите заметки в таблице: имя блока, зачем он пользователю, будущий HTML-тег, токены, состояния.
- После каждого раздела выполняйте короткую самопроверку и объясняйте решение своими словами.
- Не оценивайте этап по красоте будущей вёрстки: сейчас проверяются умение читать макет и готовность передать его в HTML.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа stage-00-figma

1.1. Тема, цель и место этапа в разработке

1.2. Состав учебных материалов

1.3. Планируемые результаты

1.4. Запуск и первичное исследование

2. Основы файла Figma

2.1. Зачем верстальщику Figma

2.2. Файл, страница, фрейм и слой

- 2.3. Инспектор свойств
- 2.4. Dev Mode и границы автоматического CSS
- 3. Макет как спецификация
 - 3.1. Почему скриншот не заменяет макет
 - 3.2. Иерархия слоёв и карта страницы
 - 3.3. Имена слоёв и заполнители
- 4. Сетка, контейнер и ритм
 - 4.1. Двенадцатиколоночная сетка
 - 4.2. max-width и край холста
 - 4.3. Шаг 8px
- 5. Отступы: gap, padding, margin
- 6. Токены дизайн-системы
 - 6.1. Цвет, HEX и повтор
 - 6.2. Типографика
 - 6.3. Радиус и тень
 - 6.4. Токены учебного проекта CORAL
- 7. Компоненты, варианты и Auto Layout
- 8. Состояния интерфейса и смысл будущего тега
- 9. Карта макета CORAL
 - 9.1. Главная страница сверху вниз
 - 9.2. Каталог, категории и товары
 - 9.3. Чего нет на холсте
- 10. Иконки, изображения и экспорт
- 11. Доступность уже на этапе макета
- 12. Передача макета в HTML, CSS и JavaScript
- 13. Практический разбор и упражнения
- 14. Типичные ошибки и способы исправления
- 15. Контрольные вопросы
- 16. Итоговая самопроверка по практическому примеру
- Заключение
- Источники и рекомендуемые материалы

1. Назначение этапа stage-00-figma

1.1. Тема, цель и место этапа в разработке

Название темы методических указаний: «Макет Figma как техническое задание многостраничного интернет-магазина». Исходный этап моделирует работу с Coral.fig до написания HTML. Такой порядок соответствует принципу разделения ответственности: сначала фиксируют содержание, ритм и визуальную систему, затем описывают структуру документом, потом оформляют её стилями и добавляют поведение.

Цель этапа — перевести визуальный макет в логическую систему блоков и значений. Разработчик должен определить, где располагаются шапка, навигация, герой, категории, карточки, баннер, подписка и подвал; какие цвета и шрифты являются токенами; какие состояния спрятаны за кадром default.

ВАЖНО

Figma — инструмент проектирования интерфейса, а не среда вёрстки. Макет описывает, как страница должна выглядеть и из каких частей состоять, но сам по себе не создаёт HTML-документ, не задаёт доступные имена разделов и не реализует отправки формы.

1.2. Состав учебных материалов

Файл / место	Роль в проекте	Что изучается
Coral.fig	Исходный макет магазина	Фреймы, слои, компоненты, Inspect
Онлайн-копия Figma	Доступ без локального файла	Тот же холст Design ~1920 px
design/tokens.md	Снятые токены	Цвета, шрифты, имена компонентов
docs/course/00-figma.md	Карта блоков	Соответствие слоёв будущему HTML
guides/00-figma.md	Пошаговая инструкция	Как снимать спецификацию руками
stage-01-html/	Следующий этап	Куда уходит карта блоков

1.3. Планируемые результаты

Студент должен знать назначение файла, страницы, фрейма и слоя; смысл сетки и шага отступов; роль токена; отличие компонента от разовой группы; различие состояний default, hover, sale и disabled.

Студент должен уметь:

открыть учебный макет и найти основной фрейм главной страницы;

выписать блоки сверху вниз и сопоставить их с будущими header, nav, main, section, article, footer;

снять 5–8 цветов и шкалу кеглей, записать HEX и размер в px;

найти повторяющийся компонент card-product и его варианты;

отличить gap Auto Layout от ручного сдвига слоя;

объяснить, какие части макета станут HTML, какие — CSS, а какие потребуют JavaScript;

найти скрытые состояния и учебные заготовки, которых нет на холсте (модалки входа и товара).

1.4. Запуск и первичное исследование

Откройте репозиторий Coral_Example и файл Coral.fig либо онлайн-макет по ссылке курса. Найдите страницу Design и самый высокий фрейм шириной около 1920 px — это главная. Пройдитесь по дереву слоёв сверху вниз: top-header, menu, top-main, brand, hot-category, grid-products, brand-banner, slider-products, Instagram, newsletter, footer. Выделите кнопку Shop now и карточку card-product. Запишите цвет, кегль, padding и наличие вариантов. Сверьте свои записи с design/tokens.md: Primary #9E5CF2, текст #1E2832, обводка #E3E3E3. Отметьте, что отдельных фреймов Login и Product в макете нет — это сознательный зазор курса.

ОБРАТИТЕ ВНИМАНИЕ

Без Dev Mode и без панели Inspect макет остаётся картинкой. «Красиво выглядит» не означает, что спецификация считана: на этапе 0 проверяют числа, имена слоёв, повтор компонентов и пользовательский смысл блока.

2. Основы файла Figma

2.1. Зачем верстальщику Figma

Figma — совместный редактор интерфейсов в браузере и в настольном приложении. Для курса важны не навыки «рисовать с нуля», а навыки читать готовый файл как договор между дизайном и кодом. Верстальщик снимает ритм, токены, компоненты и состояния. Дизайнер мог заложить их аккуратно или оставить учебные шероховатости; и то и другое входит в задание.

Онлайн-копия учебного макета опубликована в Figma-файле курса (ссылка — в разделе источников). Локальная копия лежит в корне репозитория как Coral.fig. Содержимое одно и то же: магазин одежды и аксессуаров CORAL, а не туристический сайт.

2.2. Файл, страница, фрейм и слой

Понятие	Пример в CORAL	Пояснение
Файл	Coral.fig	Весь проект дизайна.
Страница	Design	Вкладка внутри файла; в учебном макете основная страница одна.
Фрейм	холст ~1920×6110	Рабочая область экрана или артборд.
Слой	top-header, card-product	Именованный объект в дереве.
Группа	hero-gallery	Связка слоёв без семантики компонента.
Компонент	card-product, button	Повторяемый элемент с вариантами.

Картинка показывает, как страница выглядит. Дерево слоёв показывает, как она устроена: что является шапкой, что карточкой, что повтором. Именованный слой icon / search полезен тем, что понятно назначение ассета. Он не становится URL API и ничего не шифрует — это человекочитаемое имя.

2.3. Инспектор свойств

Правая панель Figma в режиме просмотра и в Dev Mode показывает размеры фрейма, отступы Auto Layout, цвет заливки, эффекты, шрифт, кегль, межстрочный интервал и экспорт. Это основной инструмент этапа 0. Две карточки на скриншоте могут выглядеть «с одинаковым зазором», тогда как у одной Gap равен 24 px, а у другой слой сдвинут вручную на 21 px. Без инспектора это не отличить — в вёрстке ряд «поплывёт».

```
Desktop 1920
├── header / top-header
│   ├── search
│   ├── logo
│   └── account + cart
├── nav / menu
├── hero / top-main
└── grid-products
    ├── card-product ← повтор
    ├── card-product
    └── card-product
```

Пример 1 — Дерево слоёв главной страницы CORAL

2.4. Dev Mode и границы автоматического CSS

Dev Mode упрощает съём значений и иногда предлагает фрагмент CSS. Это подсказка, а не готовая вёрстка. Автоматический CSS не знает семантических тегов, не строит иерархию заголовков, не связывает label с input и не заменяет дизайн-токены именованными переменными проекта. На курсе его используют как линейку, а каркас пишут руками на этапе 1.

ВАЖНО

Скопированный из Figma CSS не равен эталону этапа 2. Он не отменяет карту блоков и не освобождает от проверки контраста, фокуса и смысла тега.

3. Макет как спецификация

3.1. Почему скриншот не заменяет макет

Скриншот даёт общее впечатление. Он не даёт точный padding, кегль, межстрочный интервал, имена слоёв и состояния, которых на кадре нет: hover, disabled, открытое меню, вариант Sale. Поэтому верстать «только по картинке» нельзя: получится похожий кадр, но не спецификация.

Источник	Что даёт	Чего не даёт
PNG / скрин	Композицию и общее настроение	Точные отступы и скрытые состояния
Файл Figma	Числа, дерево, компоненты	Готовый HTML и поведение
Прототип	Сценарий переходов	Серверную логику и валидацию

3.2. Иерархия слоёв и карта страницы

Карта страницы — это перевод дерева слоёв на язык будущих областей документа. На этапе 0 ещё не пишут теги, но уже решают, какой смысл у блока. Повтор card-product — один компонент, а не восемь уникальных картинок. В коде это будет один шаблон карточки внутри списка.

3.3. Имена слоёв и заполнители

«Lorem ipsum» и близкие заглушки — заполнитель. Контент заменят. Это не юридический текст и не часть протокола. В герое CORAL учебный абзац содержит опечатки: ans, differnt, barands. На этапе вёрстки текст копируют как в макете, потому что макет — ТЗ. В живом магазине копирайты правят отдельно.

Тот же принцип относится к «Shoping» на слое корзины и к записи «© 2022 Coral , Inc.» в подвале. Студент учится следовать спецификации, а не улучшать её молча.

САМОПРОВЕРКА

Найдите в макете слой card-product. Сколько экземпляров вы видите на главной? Есть ли у компонента варианты Hover, Discount или Hot? Почему восемь разных классов в будущем CSS были бы хуже одного повторяемого блока?

4. Сетка, контейнер и ритм

4.1. Двенадцатиколончатая сетка

Колонки и желоб нужны, чтобы выравнивать блоки по одной системе. Они не уменьшают вес картинок, не заменяют HTML и не отключают адаптив. Число 12 удобно тем, что делится на 2, 3, 4 и 6 без остатка. Браузеры не «понимают только 12»: это соглашение макета, которое потом выражают сеткой CSS.

Доли из 12	Что получается в CORAL
12	Герой и баннер на всю ширину контейнера
6 + 6	Две равные колонки, фото + текст героя
4 + 4 + 4	Три колонки подвала
3 + 3 + 3 + 3	Четыре карточки каталога в ряд
8 + 4	Широкое фото и текстовая колонка

Желоб в коде — это gap или padding колонки. Общая ширина контента — контейнер. В учебном проекте контейнер около 1320 px.

4.2. max-width и край холста

Холст макета равен 1920 px, но строка текста не должна растягиваться на ультрашироком мониторе до 2500 px: её невозможно читать. Поэтому фиксируют max-width контейнера. Это не запрет мобильной версии, не отключение масштабирования и не замена HTML.

```
.container {
```

```
width: min(1320px, calc(100% - 40px));
margin-inline: auto;
}
```

Пример 2 — Контейнер, который потолок держит на широком экране и оставляет поля на узком

На узком экране сработает 100% минус поля. На широком сработает потолок 1320 px. Край холста 1920 px при этом остаётся справочным кадром desktop-first, а не обязательной шириной страницы.

4.3. Шаг 8px

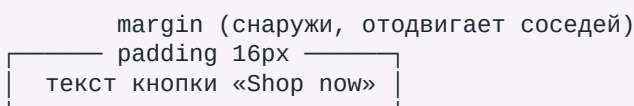
«Отступы 8px-системой» означает: размеры кратны шагу 4 / 8 / 16 / 24 / 32. Это не требование «шрифт только 8 px» и не один отступ на весь сайт. Система убирает спор о «почти одинаковых» тройках пикселей и держит вертикальный ритм.

Задача	Берём	Не берём
Воздух в карточке	16 или 24	17, 19, 23
Зазор между карточками	24	21 «на глаз»
Высота кнопки	padding 8+8 или 16+16	13 сверху и 11 снизу

5. Отступы: gap, padding, margin

Spacing between items в списке карточек — это gap между карточками, не z-index и не charset. В Figma это поле Auto Layout Gap. В CSS — свойство gap у flex или grid. Внешний отступ всего списка — уже margin родителя.

То, что внутри рамки блока до контента, ближе к padding. Margin отодвигает блок от соседей. Путать их — частая причина «плывущей» сетки: студент увеличивает margin карточки, хотя в макете задан gap списка.



```
margin (снаружи, отодвигает соседей)
padding 16px
текст кнопки «Shop now»
```

Пример 3 — Внутренний и внешний отступ кнопки

```
.product-list {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 24px; /* spacing between items */
}
.btn {
  padding: 16px 32px; /* внутренний */
  margin-top: 24px; /* внешний, до соседа */
}
```

Пример 4 — Перенос Gap и Padding из Auto Layout в CSS

6. Токены дизайн-системы

6.1. Цвет, HEX и повтор

Токен — именованное значение, которое переиспользуют: `primary`, `black`, `stroke`. Не случайный градиент и не «просто HEX без имени». Один `primary` обслуживает кнопки, бейджи и ховер ссылок. Сменили токен — поменялся акцент во всём интерфейсе.

Цвет `#1E2832` в CSS пишут как `#1e2832`. Регистр не важен: это тот же sRGB. Не обязательно переписывать значение в `rgb()` с другими числами, если каналы совпадают.

```
:root {
  --color-primary: #9e5cf2;
  --color-black: #1e2832;
  --color-stroke: #e3e3e3;
}
.btn:hover { background: var(--color-primary); }
.badge     { background: var(--color-primary); }
```

Пример 5 — Один токен на несколько ролей

6.2. Типографика

Interline (line-height) задаёт межстрочный интервал текста. Он не заменяет `margin` у картинок и не сжимает PNG. Значение 1.5 при кегле 16 px даёт шаг строки 24 px — как раз кратно 8px-системе. Плотный интервал у абзаца делает «кирпич», слишком свободный — рассыпает текст.

Частая причина расхождения макета и кода: в файле стоит локальный или платный шрифт, в вебе — другой набор. В CORAL логотип набран Amstelvar Alpha. Без лицензии в браузере его может не быть; на этапе вёрстки ставят доступный гротеск и принимают отличие. Заголовки Roboto и текст Open Sans подключают через `@font-face` или каталог веб-шрифтов.

Выравнивание по базовой линии связано с ритмом строк. Иконка стрелки рядом с «Shop now» должна сидеть на той же линии, а не «по центру бокса вслепую». В CSS это часто `inline-flex` и `align-items: baseline`.

6.3. Радиус и тень

Скругление лучше хранить одним токеном для кнопок и полей, а не случайным числом на каждый блок. Иначе кнопка 6 px, поле 11 px и карточка 4 px визуальнo разъедутся. Тень в вебе обычно делают через `box-shadow`. Тёга shadow нет. В Figma эффект снимают как X, Y, blur, spread и цвет с прозрачностью.

6.4. Токены учебного проекта CORAL

Имя в макете	HEX	Назначение
Primary	#9E5CF2	Акцент, кнопки Shop now, бейджи
Primary 75/50/25/10/5	оттенки фиолетового	Ховер и светлые фоны
Black	#1E2832	Основной текст
White	#FFFFFF	Фон страницы

Dark	#000000	Баннер Zara, тёмные блоки
Stroke	#E3E3E3	Линии и рамки карточек

Роль	Семейство	Пример из макета
Логотип	Amstelvar Alpha	CORAL
Заголовки	Roboto Medium	H1 67, H2 50, H3 37, H4 28
H5 / H6 / span	Open Sans SemiBold	21 px / 16 px
Текст, кнопки	Roboto Regular / Light	Кнопка 18 px, body
Иконки	Font Awesome 6	Поиск, аккаунт, корзина, соцсети

Шкала заголовков в макете построена как Perfect Fourth: база 16 px, коэффициент 1,333. Это объясняет ряд 16 — 21 — 28 — 37 — 50 — 67 лучше, чем набор «красивых» случайных кеглей.

7. Компоненты, варианты и Auto Layout

Компонент в Figma ближе всего к повторяемому элементу интерфейса с вариантами. Это не таблица базы данных и не HTTP-метод. Варианты card-product: Default, Hover, Sale, Hot. В коде позже появится один блок и модификаторы, на этапе БЭМ — product-card product-card_sale.

Auto Layout держит отступы и направление (ряд или колонка) при изменении контента. Добавили длинное название — карточка не разъехалась, gap сохранился. Это прямой прообраз flexbox.

Figma Auto Layout	CSS
Direction: Horizontal	flex-direction: row
Direction: Vertical	flex-direction: column
Gap	gap
Padding	padding
Hug contents	width: fit-content
Fill container	flex: 1 / width: 100%

Style guide / UI kit — набор базовых стилей и компонентов в файле: цвета, типографика, кнопки, поля, карточки. Если в файле есть страница Components, съём токенов начинают оттуда, а не со случайного кадра лендинга. В учебном Coral.fig основные компоненты живут на том же холсте: button, card-product, item-category, item-tab, ribbon, newsletter-form.

ВАЖНО

Не добавляйте в будущий код десять уникальных классов для десяти карточек только потому, что на холсте лежат десять экземпляров. Экземпляр компонента — повтор одной модели.

8. Состояния интерфейса и смысл будущего тега

Состояние `hover` в UI-ките показывают для интерактивных элементов, не для фонового фото и не «только для печати». В CORAL у кнопки `default` часто чёрный фон, `hover` — `primary #9E5CF2`. Это надо снять отдельно: на статичном PNG ховера нет. Рядом в коде обязателен `:focus-visible`, потому что клавиатура не умеет «навести мышь».

Вариант `Disabled` нужен, чтобы показать недоступное действие. Кнопку не удаляют из документа. Визуально снижают контраст и ставят `cursor: not-allowed`. Смысл: корзина пуста или форма невалидна — действие сейчас нельзя выполнить.

Если в макете кнопка и ссылка выглядят как один «Shop now», в коде всё равно выбирают тег по смыслу: переход по адресу — `a`, действие на странице или отправка формы — `button`. Внешний вид на этапе 0 не отменяет будущую семантику этапа 1.

В макете	Смысл	Будущий тег
Shop now → каталог	Переход	<code>a href="#products"</code>
View product	Переход на страницу товара	<code>a href="product.html"</code>
Subscribe / Sign in	Отправка формы	<code>button type="submit"</code>
Account как модалка	Действие на странице	<code>button type="button"</code>

`Hidden / off-canvas` на узком экране означает: есть отдельное состояние навигации, его надо заложить. Меню не удаляют из продукта. Если в макете нет `mobile`-фрейма, брейкпоинты всё равно появятся на этапе адаптива: 1100, 768 и 480. Прототип кликабельных фреймов помогает согласовать переходы до кода, но в учебном файле его может не быть — тогда сценарий восстанавливают по смыслу блока.

9. Карта макета CORAL

9.1. Главная страница сверху вниз

Одна основная страница `Design`, ширина 1920 px, длинный лендинг интернет-магазина. Отдельных фреймов `Login` нет.

#	Слой / смысл	HTML на этапе 1
1	top-header: поиск, логотип, Account, корзина	header
2	menu: 7 пунктов каталога	nav
3	top-main: два фото + Collections + Shop now	section.hero
4	brand: ряд логотипов	section со списком
5	hot-category: Explore new and popular styles	section категорий
6	grid-products: карточки card-product	section#products + список article
7	brand-banner: Zara, 70%, evening wear	section.banner

8	slider-products: Best sellers + «табы»	section.bestsellers
9	Instagram: Follow products and discounts	section.instagram
10	newsletter: подписка	section.newsletter + form
11	footer: колонки, соцсети, © 2022	footer

9.2. Каталог, категории и товары

Пункты меню как в макете: Jewelry & Accessories, Clothing & Shoes, Home & Living, Wedding & Party, Toys & Entertainment, Art & Collectibles, Craft Supplies & Tools.

Категории: Manto — 86 product, Pants — 200, Coat — 520, Shirt — 320. Форму «86 product» на этапе вёрстки копируют; в готовом англоязычном интерфейсе позже правят на products.

Товары, имена которых удалось снять из файла: Adicolor Classics Joggers, Nike Sportswear Futura Luxe, Geometric print Scarf, Yellow Reserved Hoodie, Basic Dress Green, Nike Air Zoom Pegasus, Nike Repel Miler, Glasses. Цены: \$63.85, \$130.00, \$53.00, \$364.00 / \$155.00 со скидкой, \$236.00, \$198.00, \$120.50, \$160.00.

Согласованность данных уже на этапе макета неидеальна: учебные изображения и категории не всегда буквально совпадают с названием. Это допустимая заглушка курса. В готовом продукте файл изображения, alt, название, тип и цена должны относиться к одной сущности.

9.3. Чего нет на холсте

Модалки авторизации в макете нет. Карточка товара как отдельный экран тоже не вынесена в собственный фрейм. Курс сознательно добавляет login.html, product.html и скрытые диалоги на следующих этапах, оформляя их тем же Primary #9E5CF2, той же типографикой и теми же кнопками shop now. Это не «вольность против макета», а продолжение дизайн-системы там, где холст молчит.

Заготовка курса	Зачем она
Страница входа и регистрации	Рабочий переход по Account без JavaScript
Страница товара	Самостоятельная сущность card-product
Скрытые dialog	Будущий ускоренный сценарий на этапах 4–5

10. Иконки, изображения и экспорт

Иконка-вектор масштабируется без мыла и перекрашивается через currentColor. Растр 24×24 на экране 3х станет размытым. Утверждение «вектор нельзя перекрасить» ложно. Фото товара — другое дело: это растр JPEG или WebP, не иконка. Вектор не всегда легче фотографии: сложный SVG может весить больше сжатого снимка. Речь идёт о масштабировании, не о любом файле.

Экспорт @2x и @3x нужен для экранов с высокой плотностью пикселей. Один CSS-пиксель там равен двум или трём физическим. Картинка 1x на таком экране мылит. В Figma: панель Export, кратность 2x / 3x. Для иконок чаще достаточно одного SVG на все плотности.

Практическая диагностика на следующем этапе совпадёт с HTML-указаниями: переименовать файл, проверить регистр, посмотреть статус в сети. На этапе 0 задача другая — понять, какие слои являются информативными изображениями товара, какие — логотипами брендов, какие — декоративной галереей Instagram.

11. Доступность уже на этапе макета

Контраст текста и фона важен из-за читаемости, не из-за SEO-титла. Ориентир WCAG для обычного текста — не ниже 4,5:1. Чёрный #1E2832 на белом проходит. Светло-серый #AAAAAA на белом не проходит, даже если «в макете так набросали подпись». Проверяют не только default: disabled часто слишком бледный, текст на фото героя тонет в изображении.

Макет не заменяет клавиатурный сценарий. Hover не равен фокусу. Если дизайнер показал только состояние мыши, верстальщик всё равно закладывает видимое :focus-visible. Имя кнопки-иконки без текста (поиск, аккаунт, корзина) позже станет aria-label; на этапе 0 это уже видно по тому, что на холсте есть только символ.

Pixel-perfect на практике — близость к макету по ритму и токенам с разумными допусками. Допустимо: ± 2 px на «воздухе», сглаживание шрифта движком системы, замена недоступного шрифта. Недопустимо: primary «примерно фиолетовый», gap 17 вместо 24, потерянные hover и disabled.

12. Передача макета в HTML, CSS и JavaScript

Этап 0 заканчивается не файлом стилей, а спецификацией. Ниже — граница ответственности, которую студент должен уметь объяснить.

Слой работы	Что забираем из Figma
HTML	Смысл блоков, порядок чтения, будущие теги, подписи форм, alt по роли изображения, ссылка или кнопка.
CSS	Сетка, контейнер, токены, типографика, радиус, тень, hover, сетка карточек, брейкпоинты.
JavaScript	Открытие модалок, сравнение паролей, живой фильтр Best sellers, фокус-ловушка диалога.
Сервер	Реальная отправка форм, поиск, учёт выбранного товара. В макете этого нет.

ОБРАТИТЕ ВНИМАНИЕ

«Табы» Best sellers в макете выглядят как фильтр, но на холсте это обычные подписи. Пока нет поведения tablist / tab / tabpanel, в HTML их размечают как ссылки или кнопки-заготовки, а не как готовые ARIA-вкладки.

13. Практический разбор и упражнения

13.1. Практикум 1. Аудит структуры Coral.fig

Цель: научиться переводить дерево слоёв в карту содержимого.

Выпишите в таблицу все крупные блоки главной в порядке появления.

Для каждого блока укажите имя слоя, зачем он пользователю и будущий HTML-тег.

Подсчитайте карточки в основном каталоге и в Best sellers.

Найдите компонент кнопки и перечислите её размеры: кегль, padding, цвет default и hover.

Отметьте элементы, которых нет на холсте, но которые появятся в курсе.

ОЖИДАЕМЫЙ ВЫВОД

Страница разделена на смысловые области, повторяемые сущности оформлены компонентами, а числа сетки и цвета уже можно перенести в таблицу токенов.

13.2. Практикум 2. Съём токенов

Цель: собрать мини-дизайн-систему, а не палитру «на глаз».

Снимите Primary, Black, White, Dark, Stroke и хотя бы один оттенок Primary.

Запишите семейства шрифтов для логотипа, заголовков, текста и иконок.

Запишите кегли H1–H4 и базы 16 px.

Сверьте таблицу с design/tokens.md и объясните расхождения, если они есть.

Сформулируйте, какие значения станут CSS-переменными на этапе 2.

13.3. Практикум 3. Разбор карточки товара

Цель: увидеть в экземпляре компонента будущий шаблон article.

Выделите один card-product и выпишите вложенность: изображение, название, категория, цена, бейдж.

Найдите варианты Hover / Sale / Hot, если они есть.

Запишите gap и padding Auto Layout.

Решите, что станет ссылкой на product.html, а что останется текстом.

Объясните, зачем позже понадобится элемент data для цены.

13.4. Практикум 4. Кнопка, ссылка и форма

Цель: не путать внешний вид и роль.

Найдите Shop now, View product, Search, Subscribe и Account.

Для каждого запишите: переход это или действие.

Для поиска и рассылки снимите состав поля: подпись видна или только placeholder.

Объясните, почему placeholder не заменит label на этапе HTML.

Предложите, как назвать скрытую кнопку-иконку поиска для скринридера.

13.5. Практикум 5. Что скрыто от кадра

Проверьте, есть ли mobile-фрейм и состояние открытого меню.

Если фрейма нет, предложите брейкпоинты курса 1100 / 768 / 480 и опишите, что произойдёт с сеткой карточек.

Опишите модалку входа, которой нет на холсте, цветами CORAL.

Объясните, почему отдельные страницы login.html и product.html всё равно нужны.

Сформулируйте три задачи JavaScript, которые макет не выполняет.

13.6. Практикум 6. Мини-аудит качества макета

Проверьте контраст основного текста и подписей на баннере.

Найдите опечатки и решите, копируете вы их как ТЗ или выносите в список правок.

Оцените alt будущих изображений Instagram: хватит ли общего Look from Instagram.

Найдите все «почти кнопки» с неясной целью.

Сформулируйте три сильные стороны макета и три направления уточнения перед вёрсткой.

14. Типичные ошибки и способы исправления

Ошибка	Почему это проблема	Исправление
Верстать только по PNG	Теряются числа и состояния	Снимать свойства в Inspect
Все блоки называть «картинка»	Нет карты смысла	Сопоставить слой и будущий тег
Цвета «примерно как в макете»	Ломается система акцента	Записать HEX в таблицу токенов
Отступы 17 / 21 / 23	Ритм рассыпается	Держать шаг 8 px
Путать gap и margin	Сетка и карточки плывут	Gap списка, padding внутри, margin снаружи
Десять уникальных карточек	Нет компонента	Один шаблон card-product
Игнорировать hover	Интерактив не специфицирован	Снять варианты компонента
Путать кнопку и ссылку	Сломается навигация и формы	Выбирать тег по смыслу
Считать Dev Mode вёрсткой	Появляется каша из чужого CSS	Брать числа, писать код самому
Править опечатки молча	ТЗ и код расходятся	Копировать текст этапа, правки — отдельно
Забыть, что нет Login-фрейма	Студент «ищет пропавшее»	Собрать экран по токенам системы
Оценивать этап по красоте CSS	Смешиваются слои работы	На этапе 0 проверять спецификацию

14.1. Чек-лист самопроверки макета

Открыт основной фрейм ~1920 px, дерево слоёв просмотрено сверху вниз.

Составлена карта блоков с будущими header, nav, main, section, article, footer.

Сняты Primary, Black, Stroke и шкала заголовков.

Найден повтор card-product и хотя бы один вариант состояния.

Записаны gap и padding ключевых списков и кнопок.

Shop now, View product, Search и Subscribe разделены по смыслу.

Зафиксированы опечатки и заглушки как часть ТЗ.

Отмечено отсутствие Login-фрейма и mobile-кадров, если их нет.

Сформулировано, что уйдёт в HTML, CSS, JavaScript и на сервер.

Заметки сохранены: notes-figma.md или таблица в тетради.

15. Контрольные вопросы

1. Что описывает макет Figma и чем его задача отличается от задач HTML, CSS и JavaScript?
2. Почему скриншот страницы не заменяет файл макета?
3. Чем фрейм отличается от слоя и от компонента?
4. Зачем верстальщику дерево слоёв, если композиция и так видна?
5. Что можно и чего нельзя ожидать от Dev Mode?
6. Почему удобна сетка из 12 колонок?

7. Зачем контейнеру max-width, если холст макета равен 1920 px?
8. Что означает «держать шаг 8 px»?
9. Чем gap отличается от padding и margin?
10. Что такое токен цвета и зачем ему имя, а не только HEX?
11. Важен ли регистр записи #1E2832 и #1e2832 в CSS?
12. Почему line-height 1.5 при кегле 16 px удобен в 8px-системе?
13. Почему веб-страница может «не совпасть» со шрифтом макета?
14. Что такое Auto Layout и какому механизму CSS он соответствует?
15. Чем компонент отличается от группы слоёв?
16. Зачем компоненту варианты Hover, Sale и Disabled?
17. Почему одинаковый вид «Shop now» не означает одинаковый HTML-тег?
18. Что означает off-canvas меню и почему его нельзя выкинуть, если в файле только desktop?
19. Зачем экспортировать растр в @2x и @3x, а иконки — в SVG?
20. Какой минимальный контраст обычного текста ориентирует WCAG?
21. Чем pixel-perfect отличается от попиксельной копии на всех устройствах?
22. Какие блоки главной CORAL станут header, nav, main и footer?
23. Почему card-product удобно мыслить как будущий article внутри li?
24. Какие токены CORAL нужно снять в первую очередь?
25. Почему опечатки героя на этапе вёрстки копируют, а не правят молча?
26. Чего нет на холсте Coral.fig, но появится в курсе?
27. Какие части Best sellers являются визуальными «табами», но ещё не поведением вкладок?
28. Что из макета уходит в CSS-переменные, а что — в семантические теги?
29. Почему hover в макете не отменяет :focus-visible в коде?
30. Как проверить, что студент прочитал макет, а не угадал картинку?

16. Итоговая самопроверка по практическому примеру

Для итоговой проверки выберите один крупный блок Coral.fig — например, карточку товара, героя или шапку — и объясните его по следующему шаблону.

Шаг анализа	Что нужно объяснить
1. Назначение	Какую задачу пользователя решает блок?
2. Границы	Какой слой или компонент является корневым контейнером?
3. Имя	Как блок назван в дереве и какое видимое название у него на холсте?
4. Вложенность	Какие слои являются родителями, детьми и соседями?
5. Токены	Какие цвета, кегли, отступы и эффекты сняты из Inspect?
6. Состояния	Есть ли hover, sale, disabled, открытое меню?
7. Навигация	Это переход, отправка формы или действие на странице?
8. Граница этапа	Что уже задано макетом, а что потребует HTML, CSS, JavaScript или сервера?

КРИТЕРИЙ ПОНИМАНИЯ

Студент не только называет инструменты Figma, но и объясняет, почему выбранный блок соответствует смыслу, как он связан с соседними областями и что с ним произойдёт на этапе HTML.

Заключение

Этап 0 демонстрирует профессионально полезный порядок работы: сначала определить содержание, ритм и визуальную систему, затем перейти к структуре документа, оформлению и интерактивности. Даже без единой строки кода проект уже содержит карту многостраничного магазина, набор токенов и список состояний, которые нельзя угадать по скриншоту.

Главный результат этапа — умение видеть за красивым холстом спецификацию. Качественное чтение макета не сводится к запоминанию кнопок Figma: оно требует согласованной иерархии блоков, точных чисел, понимания компонентов и границы клиентской разметки.

Следующий шаг курса — stage-01-html: семантический каркас index.html, login.html и product.html по карте, которая собрана в этих указаниях.

Источники и рекомендуемые материалы

Исходный учебный проект CORAL, файл Coral.fig и каталог design/.

https://github.com/kruzenshteyn/Coral_Example (дата обращения: 15.09.2026).

Онлайн-копия макета CORAL. <https://www.figma.com/design/bfB9Voiv6vjbKwcDaTakqi/Coral> (дата обращения: 15.09.2026).

Карта блоков этапа 0. https://github.com/kruzenshteyn/Coral_Example/blob/main/docs/course/00-figma.md (дата обращения: 15.09.2026).

Инструкция этапа 0. https://github.com/kruzenshteyn/Coral_Example/blob/main/docs/course/guides/00-figma.md (дата обращения: 15.09.2026).

Вводная лекция курса «Макет как ТЗ».

https://github.com/kruzenshteyn/Coral_Example/blob/main/docs/course/lectures/00-figma/lecture.md (дата обращения: 15.09.2026).

Figma Help. Components. <https://help.figma.com/hc/en-us/articles/360038662654> (дата обращения: 15.09.2026).

Figma Help. Auto layout. <https://help.figma.com/hc/en-us/articles/5739229107479> (дата обращения: 15.09.2026).

Figma Help. Dev Mode. <https://help.figma.com/hc/en-us/articles/15023124644247> (дата обращения: 15.09.2026).

W3C Web Accessibility Initiative. Contrast and Structure Tutorials.

<https://www.w3.org/WAI/tutorials/> (дата обращения: 15.09.2026).

WHATWG. HTML Living Standard — для следующего этапа передачи макета в документ.

<https://html.spec.whatwg.org/> (дата обращения: 15.09.2026).

Семантическая HTML-разметка
многостраничного интернет-магазина

Методические указания на примере проекта CORAL

Этап 1 — HTML

Назначение	теоретическое изучение и практический разбор HTML-каркаса
Уровень	начальный / базовый
Исходный проект	Coral_Example / stage-01-html
Формат работы	чтение кода, запуск в браузере, анализ и модификация

2026

Аннотация

Методические указания посвящены созданию семантического HTML-каркаса многостраничного интернет-магазина. Материал построен вокруг открытого учебного проекта CORAL, каталог stage-01-html: главная страница index.html, страница входа и регистрации login.html, карточка товара product.html и набор локальных изображений. Издание объясняет назначение HTML, устройство документа, смысл семантических элементов, организацию заголовков и навигации, работу ссылок, изображений, списков, форм, атрибутов доступности и скрытых диалоговых блоков. Практические фрагменты разбираются с точки зрения структуры, пользовательского сценария и качества разметки. CSS и JavaScript намеренно не используются: на первом этапе внимание сосредоточено на содержании и семантике.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет прочитать структуру HTML-документа, объяснить выбор основных элементов, собрать несколько связанных страниц, оформить доступную форму и найти типичные ошибки в учебной разметке.

Как работать с методическими указаниями

- Сначала откройте index.html в браузере и в редакторе кода; сравнивайте видимый результат со структурой исходного файла.
- Каждый пример набирайте вручную в отдельной копии проекта. После изменения обновляйте страницу в браузере.
- Используйте инструменты разработчика браузера: вкладки Elements (Элементы) и Accessibility (Доступность), если она доступна.
- После каждого раздела выполняйте короткую самопроверку и объясняйте смысл разметки своими словами.

- Не оценивайте страницу по внешнему виду на данном этапе: отсутствие оформления является особенностью stage-01-html.

Оглавление

Аннотация	2	Как работать с методическими указаниями	21.	Назначение этапа stage-01-html
41.1.	Тема, цель и место этапа в разработке	41.2.	Состав учебного каталога	
41.3.	Планируемые результаты	41.4.	Запуск и первичное исследование	52.
Основы HTML-документа	52.1.	Элемент, тег и атрибут	52.2.	Минимальный каркас
52.3.	Вложенность и дерево документа	62.4.	Комментарии и форматирование кода	
63.	Семантическая структура страницы	73.1.	Зачем нужна семантика	73.2.
Карта главной страницы	73.3.	Заголовки и доступные имена разделов	83.4.	section, article и div: критерий выбора
84.	Текст, списки и навигация	84.1.	Абзацы, списки и специальные символы	
84.2.	Относительные ссылки между страницами	94.3.	Фрагментные идентификаторы	
94.4.	Ссылка или кнопка	95.	Изображения и медиаресурсы	
105.1.	src, alt, width и height	105.2.	Информативные и декоративные изображения	
105.3.	Практическая диагностика путей	116.	Формы: ввод, подписи и браузерная проверка	
116.1.	Структура формы	116.2.	Связь label и input	116.3.
116.4.	name, required и autocomplete	126.5.	Кнопки и методы отправки	
127.	Представление товарных данных	127.1.	Карточка как article	
127.2.	Машиночитаемое значение data	137.3.	Согласованность содержимого	
138.	Подробный разбор страниц проекта	138.1.	index.html: шапка и поиск	138.2.
Hero-секция и призыв к действию	148.3.	Категории и каталог товаров	148.4.	Баннер, хиты продаж и условные «табы»
148.5.	Newsletter и подвал	148.6.	login.html: вход и регистрация	
158.7.	product.html: самостоятельная карточка товара	158.8.	Связи между страницами	
159.	Скрытые диалоговые области и граница возможностей HTML	169.1.	Атрибут hidden	
169.2.	role="dialog" и aria-modal	169.3.	Что потребуется на следующем этапе	
169.4.	Почему в проекте есть отдельные страницы	1610.	Доступность и качество разметки	
1610.1.	Доступность как часть HTML	1610.2.	Проверка клавиатурой	
1710.3.	Проверка структуры в инструментах разработчика	1710.4.	Валидация и ручная проверка	
1711.	Практический разбор и упражнения	1711.1.	Практикум 1. Аудит структуры index.html	1711.2.
1711.3.	Практикум 3. Новая карточка товара	1811.4.	Практикум 4. Расширение формы регистрации	
1811.5.	Практикум 5. Исследование hidden	1811.6.	Практикум 6. Мини-аудит доступности	
1912.	Типичные ошибки и способы исправления	1912.1.	Чек-лист самопроверки HTML-страницы	1913.
1913.	Контрольные вопросы	2014.	Итоговая самопроверка по практическому примеру	21
21	Источники и рекомендуемые материалы	21	Заключение	

1. Назначение этапа stage-01-html

1.1. Тема, цель и место этапа в разработке

Название темы методических указаний: «Семантическая HTML-разметка многостраничного интернет-магазина». Исходный этап моделирует структуру магазина CORAL до добавления визуального оформления и интерактивного поведения. Такой порядок соответствует принципу разделения ответственности: HTML описывает смысл и структуру, CSS — представление, JavaScript — поведение.

Цель этапа — перевести визуальный макет в логическую систему страниц и блоков.

Разработчик должен определить, где располагаются шапка, основная навигация, тематические разделы, карточки товаров, формы, подвал и диалоговые области. Браузер уже умеет отобразить такую страницу, построить объектную модель документа и предоставить её вспомогательным технологиям.

ВАЖНО

HTML — язык разметки, а не язык программирования. Он описывает структуру данных и связи между ними, но сам по себе не содержит алгоритмов обработки формы, фильтрации каталога или открытия модального окна.

1.2. Состав учебного каталога

Файл / папка	Роль в проекте	Что изучается
README.md	Краткая инструкция этапа	Способ запуска, перечень страниц, пояснение о CSS и скрытых модальных блоках.
index.html	Главная страница магазина	Макет всей информационной архитектуры: шапка, меню, подборки, каталог, подписка, подвал, диалоги.
login.html	Страница учётной записи	Две формы: вход и создание аккаунта; типы полей и автозаполнение.
product.html	Карточка товара	Разметка самостоятельного товара с изображением, ценой, описанием и действием.
images/	Локальные растровые ресурсы	Относительные пути, размеры изображений и альтернативный текст.

1.3. Планируемые результаты

Студент должен знать назначение базовых и семантических элементов HTML, правила вложенности, роль обязательных атрибутов, назначение доступных имён и различие между содержанием, представлением и поведением.

Студент должен уметь:

- создавать корректный каркас документа HTML5;
- строить иерархию заголовков и логические области страницы;
- использовать ссылки для переходов между страницами и фрагментами;
- размечать списки, изображения, карточки товаров и цены;
- создавать формы с подписями, типами полей и базовой браузерной валидацией;
- объяснять значение атрибутов id, class, href, src, alt, required, autocomplete, aria-label и aria-labelledby;
- находить незавершённые ссылки и элементы, которым требуется CSS, JavaScript или серверная обработка.

1.4. Запуск и первичное исследование

1. Скопируйте каталог stage-01-html в рабочую папку, сохранив вложенную папку images.
2. Откройте index.html двойным щелчком или с помощью расширения Live Server.
3. Перейдите по ссылкам Account и View product; убедитесь, что загружаются login.html и product.html.
4. Вернитесь на главную страницу и добавьте к адресу #products. Браузер прокрутит документ к элементу с id="products".
5. Откройте исходный код и сопоставьте порядок видимых блоков с порядком элементов внутри body.

ОБРАТИТЕ ВНИМАНИЕ

Без CSS браузер применяет собственные стандартные стили. «Неоформленный» вид страницы не означает, что HTML неправильный: сейчас проверяются содержание, порядок чтения и доступность.

2. Основы HTML-документа

2.1. Элемент, тег и атрибут

HTML-документ состоит из элементов. В записи `<p>Описание товара</p>` начальный тег `<p>`, текстовое содержимое и конечный тег `</p>` образуют элемент абзаца. Атрибуты уточняют поведение или смысл элемента: `<a href="product.html">` задаёт адрес ссылки, а `` связывает страницу с изображением и даёт ему текстовую альтернативу.

Понятие	Пример	Пояснение
Тег	<code><h1></code> или <code></h1></code>	Синтаксическая граница элемента.
Элемент	<code><h1>Collections</h1></code>	Законченная структурная единица документа.
Атрибут	<code>id="products"</code>	Дополнительное свойство в начальном теге.
Содержимое	Collections	Текст или вложенные элементы.
Пустой элемент	<code></code>	Элемент без дочернего содержимого; завершающий слеш в HTML5 необязателен.

2.2. Минимальный каркас

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>CORAL — Collections</title>
</head>
<body>
  <!-- Видимое содержимое страницы -->
</body>
</html>

```

Пример 1 — Каркас страниц проекта CORAL

`<!DOCTYPE html>` включает современный режим обработки документа. Корневой элемент `html` охватывает всю страницу. Атрибут `lang="en"` сообщает, что основной язык содержимого — английский. В `head` находятся метаданные, а в `body` — содержимое, с которым взаимодействует пользователь.

`meta charset="UTF-8"` задаёт кодировку и позволяет корректно отображать латиницу, кириллицу и специальные символы. `viewport` сообщает мобильному браузеру, что ширина области просмотра должна соответствовать ширине устройства. `title` формирует название вкладки, закладки и часто используется в результатах поиска.

ВАЖНО

Текст внутри title и заголовок h1 выполняют разные задачи. title идентифицирует документ в интерфейсе браузера, а h1 обозначает главную тему видимого содержимого.

2.3. Вложенность и дерево документа

Браузер преобразует HTML в дерево DOM. Родительский элемент содержит дочерние элементы, а элементы одного уровня являются соседними. От корректной вложенности зависят область действия CSS, поиск узлов из JavaScript, порядок чтения экраным диктором и удобство сопровождения кода.

```
<main>
  <section aria-labelledby="products-title">
    <h2 id="products-title">Products</h2>
    <ul>
      <li><article>...</article></li>
    </ul>
  </section>
</main>
```

Пример 2 — Фрагмент дерева: main → section → ul → li → article

Здесь section является дочерним элементом main; h2 и ul — соседями внутри section; article вложен в элемент списка li. Такая структура показывает, что каталог — раздел основного содержимого, товары образуют перечень, а каждая карточка является самостоятельным материалом.

2.4. Комментарии и форматирование кода

Комментарии вида `<!-- ... -->` не отображаются на странице, но объясняют намерение автора. В index.html комментарии разделяют крупные блоки: шапку, меню, hero-секцию, категории, товары, баннер, подписку, подвал и модальные окна. Это полезно для обучения, однако комментарий не заменяет семантический элемент.

Отступы и переносы строк в большинстве случаев не меняют отображение, но делают иерархию заметной. Рекомендуется использовать один стиль отступов во всём проекте, например два пробела на уровень вложенности.

САМОПРОВЕРКА

Найдите в index.html закрывающий тег `</main>`. Какие элементы расположены после него и почему они не входят в основное содержимое?

3. Семантическая структура страницы

3.1. Зачем нужна семантика

Семантический элемент сообщает назначение содержимого. Например, nav обозначает навигационный блок, article — самостоятельный материал, footer — подвал. Универсальные div и span не несут такого значения: они нужны, когда подходящего специализированного элемента нет или требуется техническая группировка.

Семантика помогает браузерам, поисковым системам, экраным дикторам и разработчикам. Она не гарантирует красивый внешний вид, но создаёт устойчивый каркас, к которому позднее подключаются стили и сценарии.

Элемент	Смысл	Пример в CORAL
header	Вводная область страницы или раздела	Шапка с поиском, логотипом и пользовательскими ссылками.
nav	Основная группа навигационных ссылок	Меню каталога и две группы ссылок в подвале.
main	Уникальное основное содержимое страницы	Все центральные разделы index.html.
section	Тематический раздел с доступным именем	Hero, категории, товары, баннер, подписка.

Элемент	Смысл	Пример в CORAL
article	Самостоятельная единица, пригодная для отдельного использования	Карточка товара и товар на product.html.
footer	Заключительная и служебная область	Описание, ссылки, копирайт, возврат вверх.
div	Нейтральный контейнер	header-top, hero-gallery, hero-content, modal-dialog.

3.2. Карта главной страницы

Порядок	Область	Главный элемент	Назначение
1	Шапка	header	Поиск, логотип, учётная запись, корзина, меню каталога.
2	Первый экран	section.hero	Главное сообщение и переход к товарам.
3	Бренды	section.brands	Перечень представленных марок.
4	Категории	section.categories	Переходы к товарным группам.
5	Товары	section#products	Основной каталог карточек.
6	Рекламный блок	section.banner	Продвижение отдельной коллекции.
7	Хиты продаж	section.bestsellers	Сокращённый товарный список и ссылки-фильтры.
8	Социальная подборка	section.instagram	Галерея изображений.
9	Подписка	section.newsletter	Форма ввода e-mail.
10	Подвал	footer	Служебная навигация и сведения.
11–12	Диалоги	div[role="dialog"]	Скрытые заготовки окон входа и товара.

3.3. Заголовки и доступные имена разделов

На главной странице используется один h1 — Collections. Основные тематические разделы получают h2, а названия товаров внутри каталога — h3. Такая иерархия позволяет быстро просмотреть структуру документа и перемещаться по заголовкам с помощью вспомогательных технологий.

```
<section class="products" id="products"
  aria-labelledby="products-title">
  <h2 id="products-title">Products</h2>
  ...
</section>
```

Пример 3 — Заголовок одновременно видим и связан с разделом `aria-labelledby` содержит идентификатор элемента, текст которого становится доступным именем раздела. В `section.brands` видимого заголовка нет, поэтому используется `aria-label="Brands"`. Первый подход предпочтителен, когда заголовок полезен всем пользователям.

ВАЖНО

Не выбирайте h1–h6 по размеру шрифта. Уровень отражает место темы в структуре документа; внешний вид позднее настраивается CSS.

3.4. section, article и div: критерий выбора

Используйте `section`, когда группа образует самостоятельную тему и может получить заголовок. Используйте `article`, когда фрагмент имеет смысл вне текущей страницы: карточка

товара, публикация, новость, отзыв. Используйте `div` для технической группировки, когда отдельного смыслового раздела нет.

Вопрос	Да	Нет
У группы есть собственная тема и заголовок?	Рассмотрите section.	Перейдите к следующему вопросу.
Фрагмент можно вынести или повторно использовать отдельно?	Рассмотрите article.	Используйте <code>div</code> или не создавайте контейнер.
Элемент нужен только для раскладки нескольких частей?	Обычно <code>div</code> .	Проверьте наличие более точного элемента.

В footer-about применён section без явного заголовка. Для учебного улучшения можно либо добавить заголовок (в том числе визуально скрываемый CSS на следующем этапе), либо заменить section на `div`, если это исключительно композиционный контейнер.

4. Текст, списки и навигация

4.1. Абзацы, списки и специальные символы

Абзац `p` предназначен для связного текста и коротких самостоятельных строк. Однородные элементы меню, категории, карточки и социальные ссылки представлены списками `ul/li`. Список сообщает, что элементы относятся к одной группе; визуально маркеры позже можно убрать стилями.

Символ `&` в HTML-коде часто записывается как сущность `&`. В пункте Jewelry & Accessories браузер показывает «Jewelry & Accessories», не принимая амперсанд за начало некорректной сущности.

```
<nav class="menu" aria-label="Catalog">
  <ul>
    <li><a href="#products">Jewelry &amp; Accessories</a></li>
    <li><a href="#products">Clothing &amp; Shoes</a></li>
  </ul>
</nav>
```

Пример 4 — Навигация как именованная область со списком ссылок

4.2. Относительные ссылки между страницами

Адрес `index.html` или `product.html` является относительным: браузер ищет файл относительно текущего каталога. Это удобно для проекта, который можно открыть локально или разместить на сервере без переписывания абсолютного домена.

Запись href	Результат
<code>index.html</code>	Переход на главную страницу в том же каталоге.
<code>login.html</code>	Переход к странице учётной записи.
<code>product.html</code>	Переход к карточке товара.
<code>#products</code>	Переход к элементу <code>id="products"</code> на текущей странице.
<code>index.html#products</code>	Переход к разделу Products на главной из другой страницы.
<code>#</code>	Заглушка: адрес не определён; для готового проекта нежелательна.

4.3. Фрагментные идентификаторы

Ссылка `href="#products"` находит элемент с `id="products"`. Идентификатор должен быть уникальным в пределах страницы. Такой переход работает без JavaScript, поддерживает закладки и сохраняется в адресной строке.

НАЙДЕННАЯ ОСОБЕННОСТЬ ПРОЕКТА

Ссылка «Scroll to top» использует href="#top", но элемента с id="top" в index.html нет. Исправление: добавить id="top" корневой верхней области, например <header id="top">, либо использовать корректный существующий идентификатор.

```
<header class="header" id="top">
```

```
...
```

```
</header>
```

```
<a href="#top">Scroll to top</a>
```

Пример 5 — Исправленная ссылка возврата к началу страницы

4.4. Ссылка или кнопка

Ссылка <a> переводит пользователя к другому адресу или фрагменту. Кнопка <button> запускает действие: отправляет форму, открывает окно, переключает состояние. В проекте Shop now и View product являются ссылками, потому что ведут к разделу или странице. Sign in — кнопка отправки формы.

Ситуация	Элемент	Обоснование
Открыть product.html	<a>	Меняется адрес документа.
Перейти к #products	<a>	Меняется фрагмент адреса.
Отправить форму	<button type="submit">	Выполняется действие формы.
Закрыть модальное окно	<button type="button">	Будущий сценарий изменит состояние интерфейса.

5. Изображения и медиаресурсы

5.1. src, alt, width и height

```

```

Пример 6 — Изображение товара

src содержит относительный путь: сначала папка images, затем имя файла. alt передаёт смысл изображения в текстовой форме, если картинка не загрузилась или используется экранный диктор. width и height задают внутренние размеры и позволяют браузеру заранее зарезервировать место, уменьшая скачки макета при загрузке.

Для карточки товара альтернативный текст может совпадать с кратким названием модели. Для атмосферных изображений hero-секции в проекте даны описания сцены. Логотип Zara получает alt="Zara", потому что изображение передаёт название бренда.

ВАЖНО

alt описывает назначение изображения в конкретном контексте, а не механически перечисляет всё, что видно. Не начинайте текст словами «изображение» или «картинка»: экранный диктор уже сообщает тип элемента.

5.2. Информативные и декоративные изображения

Тип	Как оформить	Пример
Информативное	Краткий осмысленный alt	Фотография конкретного товара.

Тип	Как оформить	Пример
Функциональное внутри ссылки	alt описывает цель перехода	Логотип, ведущий на главную.
Декоративное	alt=""	Текстура или орнамент без самостоятельного смысла.
Сложное	Краткий alt + развернутое описание рядом	Инфографика или схема с данными.

В Instagram-подборке alt="Look from Instagram" очень общий. Для учебного улучшения полезно написать отличающиеся описания, если изображения несут информацию. Если подборка служит только декором и рядом нет важного содержания, допустим пустой alt, но решение принимается по цели блока.

5.3. Практическая диагностика путей

1. Переименуйте временную копию product-green-dress.jpg и обновите страницу: появятся значок ошибки и альтернативный текст.
2. Верните имя файла. Проверьте регистр букв: на многих серверах Dress.jpg и dress.jpg считаются разными файлами.
3. Переместите HTML-файл в подкаталог и определите, как должен измениться путь к images/.
4. Откройте сетевую вкладку инструментов разработчика и найдите запрос изображения и его статус.

6. Формы: ввод, подписи и браузерная проверка

6.1. Структура формы

Форма объединяет элементы ввода и описывает способ отправки данных. action указывает обработчик, method — HTTP-метод. В учебном этапе action="#" является заглушкой: браузер выполняет стандартную отправку, но сервер не принимает и не сохраняет данные.

```
<form action="#" method="post">
<p>
  <label for="login-email">Email</label>
  <input
    id="login-email"
    type="email"
    name="email"
    autocomplete="email"
    required
  />
</p>
  <button type="submit">Sign in</button>
</form>
```

Пример 7 — Поле e-mail на странице входа

6.2. Связь label и input

Значение for у label совпадает с id поля. Благодаря этому нажатие по подписи переводит фокус в input, а вспомогательная технология корректно объявляет назначение поля. Визуальный placeholder не заменяет label: он исчезает при вводе и может иметь недостаточную контрастность.

САМОПРОВЕРКА

Щёлкните по слову Email на странице login.html. Если фокус перешёл в поле, связь label–input работает.

6.3. Типы полей и их назначение

type	Назначение	Поведение браузера
search	Поисковый запрос	Может показывать кнопку очистки и специальную экранную клавиатуру.
email	Адрес электронной почты	Проверяет базовую форму адреса перед отправкой.
password	Пароль	Маскирует ввод; не шифрует данные самостоятельно.
text	Обычный короткий текст	Не применяет предметную проверку.

type="password" влияет только на отображение символов. Защита передачи требует HTTPS, а хранение и проверка пароля выполняются сервером. HTML-разметка не реализует аутентификацию.

6.4. name, required и autocomplete

Атрибут name задаёт имя пары «ключ–значение» при отправке формы. required запрещает штатную отправку пустого обязательного поля. autocomplete помогает менеджеру паролей и браузеру выбирать подходящие сохранённые данные.

autocomplete	Применение в проекте
email	Адрес входа, регистрации и подписки.
current-password	Существующий пароль при входе.
new-password	Новый пароль при создании учётной записи.
name	Имя пользователя при регистрации.
off	Поисковая строка, где автозаполнение отключено автором.

Форма регистрации содержит два поля нового пароля, но HTML не сравнивает их значения. Для проверки совпадения нужен JavaScript и повторная серверная проверка. Серверная проверка обязательна, потому что ограничения в браузере можно обойти.

6.5. Кнопки и методы отправки

button type="submit" отправляет ближайшую форму. button type="button" не делает этого и подходит для действий, контролируемых JavaScript. Явное указание type защищает от случайной отправки формы после перестройки разметки.

ВАЖНО

Клиентская валидация повышает удобство, но не является механизмом безопасности. Сервер должен повторно проверять все полученные значения.

7. Представление товарных данных

7.1. Карточка как article

```

<li>
<article class="product-card">
  <a href="product.html">
    
    <h3>Nike Sportswear Futura Luxe</h3>
  </a>
  <p>Bag</p>
  <p><data value="130.00">$130.00</data></p>
  <a href="product.html">View product</a>

```

```
</article>
</li>
```

Пример 8 — Полная структура карточки товара

li показывает принадлежность карточки к перечню товаров. article делает карточку самостоятельной единицей. Ссылка вокруг изображения и h3 объединяет два кликабельных представления одного перехода. Категория и цена остаются текстовыми данными, а отдельная ссылка View product явно называет действие.

7.2. Машиночитаемое значение data

Элемент data связывает видимый текст с машинно-обрабатываемым значением.

Пользователь видит \$130.00, а value="130.00" содержит число без символа валюты. Это может пригодиться для сортировки и обработки сценариями, хотя валюта всё равно должна храниться отдельно или быть однозначно известна.

```
<p>
  <s><data value="364.00">$364.00</data></s>
  <data value="155.00">$155.00</data>
</p>
```

Пример 9 — Старая и новая цена товара со скидкой

Элемент s означает, что прежняя цена больше не актуальна. Он не сообщает причину изменения; контекст Sale рядом с карточкой объясняет скидку. Для финансовых данных в реальном приложении дополнительно продумываются валюта, локализация и структурированные данные.

7.3. Согласованность содержимого

Семантическая корректность включает не только теги, но и согласование данных. В учебном файле встречаются условные изображения и категории, не всегда буквально совпадающие с названием товара. Это допустимо как заглушка этапа, однако в готовом продукте текст, alt, файл изображения, категория и цена должны относиться к одной сущности.

Проверяемая пара	Контрольный вопрос
img alt ↔ изображение	Описывает ли alt именно показанный товар?
h3 ↔ ссылка	Понятно ли название цели перехода?
категория ↔ товар	Соответствует ли Dress, Bag, Shoe фактическому типу?
видимая цена ↔ value	Совпадают ли числа и десятичный разделитель?
бейдж ↔ состояние	Подтверждают ли Sale или Hot реальные данные?

8. Подробный разбор страниц проекта

8.1. index.html: шапка и поиск

Шапка объединяет поисковую форму, логотип, пользовательские ссылки и основное меню каталога. class="header" и дочерние классы подготовлены для будущего CSS. Поисковая форма получает role="search", поэтому распознаётся как область поиска.

```
<form class="search-form" action="#" method="get" role="search">
  <label for="site-search">Search</label>
  <input id="site-search" type="search" name="q"
    placeholder="Search" autocomplete="off" />
  <button type="submit">Search</button>
</form>
```

Пример 10 — Поисковая форма

Метод GET соответствует поиску: запрос обычно может быть отражён в адресе и повторён. В текущем проекте action="#" не ведёт к поисковому сервису, поэтому это структурная заготовка.

8.2. Него-секция и призыв к действию

Hero-секция содержит два изображения, главный заголовок, короткий текст и ссылку Shop now. Связь `aria-labelledby="hero-title"` даёт разделу имя Collections. Ссылка ведёт к `#products` и работает без JavaScript.

```
<section class="hero" aria-labelledby="hero-title">
  <div class="hero-gallery">...</div>
  <div class="hero-content">
    <h1 id="hero-title">Collections</h1>
    <p>...</p>
    <a class="btn" href="#products">Shop now</a>
  </div>
</section>
```

Пример 11 — Логическая структура первого экрана

РЕДАКТОРСКАЯ ПРОВЕРКА

В исходном промотексте есть опечатки: ans, differnt, barands. Семантика не отменяет вычитку содержимого. Исправленный вариант: “You can explore and shop many different collections from various brands here.”

8.3. Категории и каталог товаров

Категории представлены одним списком ссылок. Каждая ссылка включает изображение и два `span`: название категории и количество товаров. `span` подходит для коротких частей строки без самостоятельной блочной семантики. Форму “86 product” в готовом англоязычном интерфейсе следует заменить на “86 products”.

Каталог Products использует `ul.product-list`. Каждая карточка помещена в `li` и `article`. Это масштабируемая модель: новый товар добавляется как новый элемент списка, а структура карточки остаётся одинаковой.

8.4. Баннер, хиты продаж и условные «табы»

Рекламный баннер оформлен как `section` с изображением, логотипом, процентом, заголовком и ссылкой. Заголовок по содержанию больше похож на описание коллекции; для готовой версии его можно сделать короче, а подробность перенести в `p`.

В Best sellers список `class="tabs"` содержит обычные ссылки на `#products` и `role="list"`. Это не интерактивные вкладки: они не меняют набор товаров и не используют модель `tabpanel/tab/tabpanel`. Комментарий в исходнике описывает будущий фильтр, который появится на этапе JavaScript.

ВАЖНО

Не добавляйте ARIA-роли вкладок только ради названия `class="tabs"`. Роль должна соответствовать реально реализованному клавиатурному поведению.

8.5. Newsletter и подвал

Форма подписки повторно использует `type="email"`, `label`, `autocomplete` и `required`. Это хороший пример единообразия. Подвал содержит описание, социальные ссылки, две навигационные области, копирайт и возврат вверх.

Несколько ссылок используют `href="#"` и являются заглушками. В учебном прототипе они показывают будущие точки навигации, но при проверке готового продукта такие адреса должны считаться незавершёнными.

8.6. login.html: вход и регистрация

Страница имеет один `h1 Account` и два `section` с `h2: Sign in` и `Create account`. `aria-labelledby` связывает каждый раздел с его заголовком. Формы не смешиваются: у каждой собственный набор полей и кнопка.

Форма	Поля	Особенность
Sign in	email, password	Используется current-password.

Форма	Поля	Особенность
Create account	name, email, password, password_repeat	Для обоих парольных полей указан new-password.

Имя password_repeat будет отправлено серверу отдельно. Совпадение полей не проверяется встроенным HTML-правилом; это задача сценария и сервера. Полезным улучшением станет minLength для учебной подсказки, но серверная политика паролей остаётся определяющей.

8.7. product.html: самостоятельная карточка товара

Весь товар помещён в article, потому что представляет самостоятельную сущность. h1 — название товара, data — цена, p — тип и описание, form с кнопкой — действие выбора.

Ссылка Back to collections возвращает на index.html.

```
<main>
<article>
  
  <h1>Basic Dress Green</h1>
  <p>Dress</p>
  <p><data value="236.00">$236.00</data></p>
  <p>Описание товара...</p>
  <form action="#" method="post">
    <button type="submit">Select product</button>
  </form>
</article>
</main>
```

Пример 12 — Смысловой каркас страницы товара

Для реальной отправки форма должна передавать идентификатор выбранного товара, например скрытым input. Сейчас кнопка показывает место будущего действия, но сервер не узнает, какой товар выбран.

8.8. Связи между страницами

Источник	Действие пользователя	Назначение
index.html	Account	login.html
index.html	Название / фото / View product	product.html
login.html	Логотип CORAL	index.html
product.html	Логотип или Back to collections	index.html
любая ссылка #products на index.html	Внутренний переход	section#products

Эта небольшая карта уже образует многостраничный сайт. Ссылки являются его маршрутизацией без JavaScript. При расширении каталога обычно создаются уникальные страницы или динамический маршрут для каждого товара.

9. Скрытые диалоговые области и граница возможностей HTML

9.1. Атрибут hidden

В конце index.html расположены modal-auth и modal-product. Атрибут hidden исключает их из обычного отображения. README предлагает временно удалить hidden, чтобы исследовать разметку. Это удобный учебный приём, но не механизм полноценного открытия и закрытия окна.

```
<div class="modal" id="modal-auth" hidden
  role="dialog" aria-modal="true"
  aria-labelledby="auth-title">
<div class="modal-dialog">
```

```

<h2 id="auth-title">Account</h2>
...
<button type="button">Close</button>
</div>
</div>

```

Пример 13 — Заготовка модального окна входа

9.2. `role="dialog"` и `aria-modal`

`role="dialog"` сообщает тип области, `aria-labelledby` задаёт имя по заголовку, `aria-modal="true"` указывает, что при открытии остальная страница не должна быть доступна как обычный контент. Эти атрибуты описывают состояние, но не реализуют его.

9.3. Что потребуется на следующем этапе

- Кнопка или ссылка, которая открывает диалог.
- Удаление `hidden` при открытии и возврат `hidden` при закрытии.
- Перенос фокуса внутрь диалога и возврат фокуса на вызывающий элемент.
- Ограничение перемещения фокуса областью открытого окна.
- Закрытие по кнопке `Close` и, обычно, по клавише `Escape`.
- Предотвращение взаимодействия с фоном и корректное восстановление состояния.

ВАЖНО

ARIA описывает интерфейс вспомогательным технологиям, но не добавляет поведения. Неработающий элемент с «правильной» ролью остаётся неработающим.

9.4. Почему в проекте есть отдельные страницы

`login.html` и `product.html` обеспечивают работоспособные переходы без JavaScript. Это надёжная базовая архитектура и пример прогрессивного улучшения: позднее модальный интерфейс может ускорить сценарий, но основные сведения и действия остаются доступны по обычным ссылкам.

10. Доступность и качество разметки

10.1. Доступность как часть HTML

Доступная разметка позволяет пользоваться страницей с клавиатуры, экранным диктором, увеличением, голосовым управлением и другими способами. В проекте уже применены важные основы: язык документа, заголовки, ориентиры, подписи форм, `alt`, типы кнопок и доступные имена разделов.

Практика	Реализация в CORAL	Оценка
Язык документа	<code>html lang="en"</code>	Корректно для англоязычного содержимого.
Главная область	<code>main</code>	Корректно, по одной на странице.
Иерархия заголовков	<code>h1 → h2 → h3</code>	В целом последовательна.
Подписи полей	<code>label[for] + input[id]</code>	Корректно связаны.
Изображения	<code>alt + width + height</code>	Хорошая база; часть <code>alt</code> можно конкретизировать.
Навигация	<code>nav + aria-label / aria-labelledby</code>	Области различимы по имени.
Диалоги	<code>role, aria-modal, aria-labelledby</code>	Структура есть, поведение ещё не реализовано.

10.2. Проверка клавиатурой

1. Не касаясь мыши, нажимайте `Tab` и отслеживайте порядок фокуса.
2. Активируйте ссылки клавишей `Enter`, а кнопки — `Enter` или `Space`.

3. Проверьте, что фокус не попадает в блоки с hidden.
4. Убедитесь, что каждая форма позволяет определить назначение поля по подписи.
5. Проверьте, что последовательность фокуса соответствует порядку чтения страницы.

10.3. Проверка структуры в инструментах разработчика

Во вкладке Elements можно раскрывать узлы DOM и наблюдать вложенность. В дереве доступности следует проверить имена навигационных областей, разделов, полей и кнопок. Если элемент не имеет понятного имени, сначала ищите решение в видимом тексте и нативном HTML; ARIA используйте для уточнения, а не для маскировки слабой структуры.

10.4. Валидация и ручная проверка

Автоматический валидатор находит синтаксические ошибки, дублирующиеся id, недопустимую вложенность и часть проблем атрибутов. Он не определит, соответствует ли фотография названию товара, понятен ли текст ссылки или реализован ли пользовательский сценарий. Поэтому техническая валидация дополняется ручным просмотром и тестированием клавиатурой.

11. Практический разбор и упражнения

11.1. Практикум 1. Аудит структуры index.html

Цель: научиться переводить исходный код в карту содержимого.

1. Выпишите в таблицу все элементы header, nav, main, section, article и footer в порядке появления.
2. Для каждого section найдите доступное имя: видимый заголовок, aria-label или aria-labelledby.
3. Подсчитайте карточки в основном каталоге и в Best sellers.
4. Найдите все ссылки на product.html и объясните, почему несколько ссылок могут вести к одной учебной странице.
5. Отметьте элементы, которые пока являются заглушками.

ОЖИДАЕМЫЙ ВЫВОД

Страница разделена на смысловые области, повторяемые сущности оформлены списками и article, а навигация уже работает без оформления и сценариев.

11.2. Практикум 2. Улучшение навигации

Цель: исправить фрагментный переход и сделать обратную ссылку точнее.

1. Добавьте id="top" к верхнему header на главной странице.
2. Проверьте работу Scroll to top.
3. В product.html измените ссылку Back to collections на index.html#products.
4. Откройте product.html и убедитесь, что обратная ссылка ведёт сразу к каталогу.
5. Объясните, почему id не следует дублировать на одной странице.

11.3. Практикум 3. Новая карточка товара

Цель: закрепить шаблон li → article и согласование данных.

1. Скопируйте одну карточку product-list.
2. Замените изображение на существующий файл из images/.
3. Измените alt, h3, категорию, видимую цену и value.
4. Проверьте, что все теги закрыты, а карточка остаётся внутри ul.
5. Объясните роль каждого уровня: ul, li, article, a, img, h3, p, data.

```
<li>
<article class="product-card">
  <a href="product.html">
    
    <h3>Classic Gold Watch</h3>
</a>
<p>Accessories</p>
<p><data value="189.00">$189.00</data></p>
<a href="product.html">View product</a>
</article>
</li>

```

Пример 14 — Вариант добавленной карточки

11.4. Практикум 4. Расширение формы регистрации

Цель: осознанно выбирать тип, имя и автозаполнение поля.

1. Добавьте поле телефона с `type="tel"`, `name="phone"` и `autocomplete="tel"`.
2. Свяжите label с input уникальным id.
3. Решите, будет ли поле обязательным, и обоснуйте решение.
4. Проверьте порядок перехода клавишей Tab.
5. Опишите, какие проверки всё равно должен выполнить сервер.

11.5. Практикум 5. Исследование hidden

1. Создайте копию index.html.
2. Удалите hidden только у modal-auth и обновите страницу.
3. Найдите диалог в дереве DOM и в дереве доступности.
4. Нажмите Close. Объясните, почему окно не закрывается.
5. Верните hidden и перечислите будущие задачи JavaScript для полноценного диалога.

11.6. Практикум 6. Мини-аудит доступности

1. Проверьте число h1 на каждой странице.
2. Составьте список навигационных областей и их имён.
3. Проверьте подписи всех input.
4. Сравните alt шести изображений Instagram и предложите более точные варианты.
5. Найдите все `href="#"` и разделите их на учебные заглушки и ссылки, которые следует заменить.
6. Сформулируйте три сильные стороны и три направления улучшения текущей разметки.

12. Типичные ошибки и способы исправления

Ошибка	Почему это проблема	Исправление
Все блоки сделаны div	Теряется смысловая структура и ориентиры.	Выбрать header, nav, main, section, article, footer по назначению.
Заголовок выбран по размеру	Нарушается иерархия тем.	Выбирать h1–h6 по уровню, размер задавать CSS.
У input нет label	Назначение поля может быть недоступно.	Связать label[for] и input[id].
Вместо ссылки используется button	Неясна навигационная семантика и нельзя открыть адрес стандартным способом.	Для перехода использовать a[href].
Вместо кнопки используется a href="#"	Создаётся ложная навигация.	Для действия использовать button с корректным type.
Пустой или файловый alt	Текстовая альтернатива не передаёт смысл.	Сформулировать краткое контекстное описание.

Ошибка	Почему это проблема	Исправление
Повторяется id	Фрагментные ссылки и ARIA-связи становятся неоднозначными.	Обеспечить уникальность id.
Ссылка ведёт к отсутствующему id	Переход не работает.	Добавить цель или изменить href.
required считают защитой данных	Ограничение можно обойти.	Повторять проверку на сервере.
ARIA используется без поведения	Объявленная модель не соответствует интерфейсу.	Сначала реализовать нативную структуру и взаимодействие.
Путь к изображению зависит от компьютера	Файл не загрузится после переноса.	Использовать относительные пути проекта.
Вид страницы оценивают до CSS	Смешиваются структура и оформление.	На HTML-этапе проверять смысл, порядок и связи.

12.1. Чек-лист самопроверки HTML-страницы

- Есть doctype, lang, UTF-8, viewport и осмысленный title.
- На странице определена одна основная область main.
- Главная тема обозначена h1; уровни остальных заголовков последовательны.
- Навигационные группы оформлены nav и различимы по имени.
- Тематические section имеют заголовок или доступное имя.
- Повторяемые сущности представлены списком, если порядок или принадлежность важны.
- Каждая ссылка имеет реальную цель, каждый id уникален.
- У информативных изображений осмысленный alt и корректный путь.
- Каждое поле формы имеет label, name и подходящий type.
- Кнопки имеют явный type.
- Документ проверен клавиатурой и валидатором.
- Тексты, категории, изображения и цены согласованы.

13. Контрольные вопросы

1. Что описывает HTML и чем его задача отличается от задач CSS и JavaScript?
2. Для чего нужен <!DOCTYPE html>?
3. Какую информацию передаёт атрибут lang?
4. Почему title не заменяет h1?
5. Что такое DOM и как вложенность HTML связана с деревом DOM?
6. В чём различие между тегом, элементом и атрибутом?
7. Когда следует использовать header, main и footer?
8. Почему не каждую группу ссылок обязательно помещать в nav?
9. Чем section отличается от div?
10. Почему карточка товара может быть оформлена элементом article?
11. Как строится иерархия h1, h2 и h3 в index.html?
12. Как aria-labelledby формирует доступное имя раздела?
13. Когда уместен aria-label и когда лучше видимый заголовок?
14. Почему меню и набор карточек представлены списками ul/li?
15. Чем относительная ссылка отличается от фрагментной?

16. Какие условия нужны для работы `href="#products"`?
17. Почему `href="#"` считается заглушкой?
18. В какой ситуации используется ссылка, а в какой — кнопка?
19. Какова роль атрибутов `src` и `alt` у `img`?
20. Зачем указывать `width` и `height` изображения?
21. Как выбрать `alt` для информативного и декоративного изображения?
22. Как связаны `for` у `label` и `id` у `input`?
23. Почему `placeholder` не заменяет `label`?
24. Что меняют `type="email"` и `required`?
25. Для чего нужны `name` и `autocomplete`?
26. Почему два парольных поля не сравниваются средствами одной HTML-разметки?
27. Что передаёт элемент `data` и чем `value` отличается от видимого текста?
28. Что означает элемент `s` в блоке цены?
29. Как действует атрибут `hidden`?
30. Почему `role="dialog"` не открывает и не закрывает модальное окно?
31. Какие операции с фокусом нужны полноценному модальному диалогу?
32. Какая ошибка есть у ссылки `Scroll to top` в исходном `index.html`?
33. Какие части проекта являются рабочей навигацией, а какие — прототипами?
34. Как проверить страницу только клавиатурой?
35. Почему автоматическая валидация не заменяет ручную проверку содержимого?

14. Итоговая самопроверка по практическому примеру

Для итоговой проверки выберите один крупный блок `index.html` и объясните его разметку по следующему шаблону:

Шаг анализа	Что нужно объяснить
1. Назначение	Какую задачу пользователя решает блок?
2. Границы	Какой элемент является корневым контейнером блока?
3. Имя	Как блок получает видимый заголовок и доступное имя?
4. Вложенность	Какие элементы являются родителями, детьми и соседями?
5. Навигация	Есть ли ссылки; куда они ведут?
6. Данные	Какие значения видит пользователь и какие доступны машине?
7. Доступность	Можно ли понять и использовать блок без мыши и изображений?
8. Граница этапа	Что уже работает в HTML, а что требует CSS, JavaScript или сервера?

КРИТЕРИЙ ПОНИМАНИЯ

Студент не только называет теги, но и объясняет, почему выбранный элемент соответствует смыслу, как он связан с соседними областями и что произойдёт в браузере.

Заключение

Этап `stage-01.html` демонстрирует профессионально полезный порядок работы: сначала определить содержание, структуру и связи, затем перейти к оформлению и интерактивности. Даже без CSS и JavaScript проект уже содержит три связанные страницы, навигацию,

каталог, изображения и формы. Семантические элементы делают исходный код читаемым и создают основу доступности.

Главный результат этапа — умение видеть за визуальным макетом систему смысловых областей и пользовательских действий. Качественный HTML не сводится к запоминанию тегов: он требует согласованной иерархии, точных текстов, корректных ссылок, доступных форм и понимания границ клиентской разметки.

Источники и рекомендуемые материалы

1. Исходный учебный проект CORAL, каталог stage-01-html.
https://github.com/kruzenshteyn/Coral_Example/tree/main/stage-01-html (дата обращения: 15.09.2026).
2. WHATWG. HTML Living Standard. <https://html.spec.whatwg.org/> (дата обращения: 15.09.2026).
3. W3C Web Accessibility Initiative. Page Structure Tutorials.
<https://www.w3.org/WAI/tutorials/page-structure/> (дата обращения: 15.09.2026).
4. W3C Web Accessibility Initiative. Forms Tutorial. <https://www.w3.org/WAI/tutorials/forms/> (дата обращения: 15.09.2026).
5. W3C Web Accessibility Initiative. Images Tutorial. <https://www.w3.org/WAI/tutorials/images/> (дата обращения: 15.09.2026).
6. W3C Markup Validation Service. <https://validator.w3.org/> (дата обращения: 15.09.2026).

Визуальное оформление интернет-магазина на CSS

Методические указания на примере проекта CORAL

Этап 2 — CSS

Назначение	перевод макета Coral.fig в каскадные стили без БЭМ
Уровень	начальный / базовый
Исходный проект	Coral_Example / stage-02-css
Формат работы	токены, сетка, flex/grid, состояния, сверка с макетом

2026

Аннотация

Методические указания посвящены визуальному оформлению семантического HTML-каркаса интернет-магазина CORAL. Материал опирается на каталог stage-02-css, токены design/tokens.md и макет Coral.fig: шапка, герой, категории, сетка товаров, баннер Zara, подписка и подвал.

Издание объясняет каскад и специфичность, блочную модель, flex и grid, позиционирование, единицы и кастомные свойства, состояния :hover и :focus-visible, работу с изображениями и шрифтами. БЭМ намеренно отложен: на этом этапе важны каскад и коробка, а не методология имён.

CSS описывает представление. Он не заменяет семантику HTML и не реализует открытие модальки или запись заказа.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет подключить файл стилей, вынести токены CORAL в :root, собрать шапку на flex и каталог на grid, задать контейнер 1320 px, оформить кнопку с видимым фокусом и объяснить, почему float для витрины больше не берут.

Как работать с методическими указаниями

- Откройте stage-02-css/index.html рядом с макетом Figma и таблицей токенов.
- Сначала вынесите цвета, контейнер и box-sizing, затем идите сверху вниз: шапка, герой, категории, карточки, баннер, подвал.
- Не прыгайте сразу в ширину 375 px — адаптив относится к этапу 7.
- Проверьте Tab: кольцо фокуса должно быть видно на ссылках и кнопках.
- Классы пока простые: header, hero, product-card, btn. Переименование — этап 3.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа stage-02-css
2. Каскад, специфичность и файл стилей
3. Блочная модель и контейнер
4. Flex, Grid и отказ от float
5. Токены, единицы и типографика
6. Состояния, картинки и декор
7. Разбор страницы CORAL
8. Практикумы
9. Типичные ошибки
10. Контрольные вопросы
11. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа stage-02-css

1.1. Тема, цель и место этапа

Тема: «Визуальное оформление интернет-магазина на CSS». Этап стоит между семантическим каркасом stage-01-html и рефакторингом имён stage-03-bem. Цель — перенести ритм и токены макета в код, не смешивая это с методологией БЭМ и не подключая Bootstrap или Tailwind.

ВАЖНО

HTML описывает смысл, CSS — представление. Если страница «некрасива» без стилей, это нормально: каркас уже проверен на этапе 1. Если без CSS ломается смысл, чинить нужно разметку, а не добавлять ещё один селектор.

1.2. Состав каталога

Файл	Роль	Что изучается
index.html	Главная с классами	Связь разметки и селекторов
css / style.css	Единый файл оформления	Каскад, токены, раскладка
images/	Растр из макета	object-fit, размеры
login.html, product.html	Связанные страницы	Повтор токенов на нескольких документах

1.3. Планируемые результаты

подключить стили через link и шрифты без инлайна на каждой карточке;
 задать :root с Primary #9E5CF2, текстом #1E2832 и контейнером 1320 px;
 включить border-box на всех элементах;
 собрать шапку на flex и каталог на grid с gap 24 px;
 описать :hover и :focus-visible у кнопки Shop now;
 объяснить специфичность и почему !important не является методом вёрстки.

1.4. Запуск

Откройте stage-02-css/index.html в браузере.
 Положите окно рядом с фреймом 1920 px из Coral.fig.
 Пройдите блоки сверху вниз и сверьте цвет, кегль и зазор карточек.
 Нажмите Tab по шапке и форме поиска.

2. Каскад, специфичность и файл стилей

Каскад складывает происхождение правила, его специфичность и порядок в файле. Авторские стили побеждают стили браузера, более специфичный селектор побеждает более общий, при равенстве побеждает то, что записано позже.

Специфичность считают по схеме (id, class, tag). Селектор `.a.b` тяжелее, чем одиночный `.b`: он выигрывает даже если `.b` написан ниже. `!important` ломает этот счёт и усложняет сопровождение — в учебном файле его не используют, кроме сброса чужого скрывания, которого на этапе 2 нет.

```
/* тяжело сопровождать */
.header nav ul li a { color: #1e2832; }
/* достаточно на этом этапе */
.menu a { color: var(--color-black); }
```

Пример 1 — Один класс вместо цепочки тегов

ВАЖНО

Инлайн `style=""` на каждой карточке отменяет смысл файла стилей: токен нельзя сменить в одном месте, каскад не работает, повтор пропадает.

3. Блочная модель и контейнер

По умолчанию `width` в CSS — ширина содержимого: `padding` и `border` добавляются снаружи. Для сетки это неудобно. Правило `box-sizing: border-box` включает `padding` в заданную ширину. Соседние вертикальные `margin` соседних блоков могут схлопываться. Поэтому ритм списка карточек задают `gap` у сетки, а не «магическим» `margin` снизу каждой карточки вперемешку с `margin` сверху следующей.

```
:root {
  --color-primary: #9e5cf2;
  --color-black: #1e2832;
  --container: 1320px;
}
*, *::before, *::after { box-sizing: border-box; }
.container {
  width: min(var(--container), calc(100% - 40px));
  margin-inline: auto;
}
```

Пример 2 — Токены и контейнер CORAL

4. Flex, Grid и отказ от float

Шапка CORAL — три зоны в ряд: поиск, логотип, аккаунт и корзина. Это задача `flex: direction row, space-between`, выравнивание по центру. Каталог — регулярная сетка из четырёх колонок и постоянного зазора. Это задача `grid`.

`Float` вырывает элемент из потока, родителя приходится «очищать». Его оставляют для обтекания текстом картинки в статье, а не для витрины магазина.

Блок CORAL	Инструмент	Почему
header-top	flex	Три зоны и вертикальное выравнивание

menu	flex	Пункты в одну строку
hero	grid	Два фото и текстовая колонка
product-list	grid 4×1fr, gap 24	Ровные ряды карточек
footer	grid 3 колонки	Описание и две группы ссылок

```
.product-list {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 24px;
}
```

Пример 3 — Каталог

5. Токены, единицы и типографика

rem считается от корня документа, em — от шрифта родителя. Для ритма страницы удобнее rem. Кастомные свойства в :root хранят цвет, радиус и ширину контейнера. calc(100% - 40px) оставляет поля 20 px, когда экран уже контейнера.

Шрифты CORAL: Roboto для заголовков и кнопок, Open Sans для части подписей. Логотип в макете набран Amstelvar Alpha; в вебе допустима замена, если лицензии нет. Подключение — @font-face или каталог веб-шрифтов, не картинка каждой буквы.

6. Состояния, картинки и декор

Hover в макете есть у кнопки и карточки, на статичном PNG его не видно. :focus-visible обязателен: клавиатура не умеет навести мышь. Псевдоэлемент ::after используют для декоративной черты или стрелки, если в разметке не нужен лишний узел.

Фото карточки должно держать высоту ряда: фиксированная высота области и object-fit: cover. Иначе разные пропорции превратят сетку в «лесенку». opacity: 0 прячет визуально, но оставляет объект в дереве доступности — для скрытия диалога этого мало.

7. Разбор страницы CORAL

Порядок работы на занятии совпадает с картой макета: шапка и поиск, герой Collections, ряд брендов, категории, сетка Products, баннер Zara 70%, Best sellers, Instagram, newsletter, подвал. На login.html и product.html повторяют те же токены и кнопку, не заводя вторую палитру.

ОБРАТИТЕ ВНИМАНИЕ

Адаптив 1100 / 768 / 480 на этапе 2 только намечают контейнером min(). Ломать сетку в одну колонку рано: иначе студент смешает раскладку desktop с мобильной и не увидит, что совпало с макетом.

8. Практикумы

8.1. Токены и контейнер

Вынесите пять цветов и ширину контейнера в :root.

Включите border-box.

Соберите .container через min() и calc().

Замените все повторённые HEX на var().

8.2. Шапка и герой

Сверстайте header-top на flex.

Соберите hero сеткой.

Сверьте padding кнопки Shop now с Inspect.

Добавьте :hover и :focus-visible.

8.3. Каталог

Задайте четыре колонки и gap 24 px.

Обрежьте фото через object-fit: cover.

Проверьте, что ряд не ломается на разной длине названия.

Сверьте цену и бейдж Sale с макетом.

9. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Цвет на глаз	Ломается система акцента	Токен в :root
content-box по умолчанию	Сетка шире контейнера	border-box
Float на витрине	Родитель схлопывается	Grid / flex
Нет :focus-visible	Клавиатура не видна	Кольцо фокуса
style на карточке	Нет каскада	Класс в файле
!important везде	Нельзя переопределить	Понизить специфичность

9.1. Чек-лист

Стили в отдельном файле, шрифты подключены.

Токены Primary, Black, Stroke совпадают с design/tokens.md.

Контейнер не шире экрана.

Четыре карточки в ряд на ширине макета.

Tab рисует видимый фокус.

10. Контрольные вопросы

1. Чем задача CSS отличается от задач HTML и JavaScript?

2. Что складывает каскад?
3. Как считают специфичность?
4. Почему `.header nav ul li a` неудобен?
5. Зачем `box-sizing: border-box`?
6. Что такое схлопывание `margin`?
7. Когда берут `flex`, а когда `grid`?
8. Почему витрину не собирают на `float`?
9. Чем `rem` отличается от `em`?
10. Зачем кастомные свойства в `:root`?
11. Что делает `min(1320px, calc(100% - 40px))`?
12. Почему `:hover` не заменяет `:focus-visible`?
13. Зачем `object-fit: cover` у фото карточки?
14. Почему инлайн-стили мешают токенам?
15. Что проверяют, положив страницу рядом с макетом?

11. Итоговая самопроверка

Выберите блок — шапку, героя или карточку — и объясните: назначение, корневой селектор, токены, раскладку, состояния, что уже работает в CSS и что останется JavaScript.

КРИТЕРИЙ ПОНИМАНИЯ

Студент не только называет свойства, но и объясняет, почему выбран `flex` или `grid` и как значение связано с Inspect макета.

Заключение

Этап 2 превращает каркас в страницу, похожую на `Coral.fig`. Главный результат — система токенов и раскладки, а не набор случайных пикселей. Следующий шаг — переименовать классы по БЭМ, не меняя картинку.

Источники

Проект CORAL, каталог `stage-02-css`. https://github.com/kruzenshteyn/Coral_Example (15.09.2026).

Лекция этапа 2. [docs/course/lectures/02-css/lecture.md](#).

Инструкция. [docs/course/guides/02-css.md](#).

MDN: каскад, специфичность, `flexbox`, `grid`, кастомные свойства, `:focus-visible`.

Макет `Coral.fig` и [design/tokens.md](#).

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

БЭМ · БЛОК · МОДИФИКАТОР

Независимые блоки БЭМ в вёрстке магазина

Методические указания на примере проекта CORAL

Этап 3 — БЭМ

Назначение	рефакторинг классов этапа 2 без смены визуала
Уровень	базовый
Исходный проект	Coral_Example / stage-03-bem
Формат работы	таблица переименований, независимость карточки, модификаторы

2026

Аннотация

Методические указания посвящены методологии БЭМ на уже свёрстанном магазине CORAL. Визуал этапа 2 сохраняется. Меняются имена: header-top становится header__top, badge-sale — product-card__badge_type_sale, выбранная карточка получает модификатор product-card_selected.

Издание объясняет блок, элемент и модификатор, независимость селекторов от дерева DOM, микс классов, запрет элемента элемента, файл на блок и причину, по которой карточку можно будет перенести в модалку.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет переименовать классы CORAL по классической схеме БЭМ, объяснить отличие элемента от модификатора, отказаться от селекторов вида header nav ul li a и проверить, что карточка выглядит одинаково в каталоге и вне его.

Как работать с методическими указаниями

- Скопируйте stage-02-css в рабочую папку этапа 3 или откройте готовый stage-03-bem.
- Составьте таблицу переименований до правок.
- Меняйте HTML и CSS парами.
- После каждого блока вырезайте карточку мысленно и спрашивайте, зависит ли она от родителя.
- Не меняйте цвета и сетку: критерий этапа — те же пиксели, другие имена.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа
2. Блок, элемент, модификатор
3. Независимость и селекторы
4. Микс, вложенность, файлы
5. Практика на CORAL
6. Практикумы
7. Типичные ошибки
8. Контрольные вопросы
9. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

Тема: «Независимые блоки БЭМ». Этап стоит после CSS и до модалок. Если начать курс с БЭМ в первый день HTML, студент заучит подчёркивания, не увидев проблему. После простых классов проблема видна: селекторы цепляются за DOM и ломаются при переносе карточки.

ВАЖНО

На курсе принята классика bem.info: `block__elem` и `block_mod`. Смешивать это с суффиксом двух дефисов `btn--primary` без договорённости нельзя.

1.2. Планируемые результаты

назвать блок, элемент и модификатор на шапке и карточке;
переименовать `header-top`, `product-card-media` и `badge-sale`;
убрать селекторы из трёх-четырёх тегов;
повесить состояние `selected` модификатором, а не инлайном;
объяснить, зачем команде одинаковые имена.

2. Блок, элемент, модификатор

Блок — независимая сущность: `header`, `menu`, `product-card`, `btn`, `footer`. Элемент — часть блока, которая вне блока не нужна: `product-card__media`, `product-card__title`, `header__top`. Модификатор — уточнение вида или состояния: `product-card_selected`, `product-card__badge_type_sale`, `btn_theme_primary`.

Два модификатора на одном узле допустимы: карточка может быть и `selected`, и `sale`. Нельзя превращать модификатор в новое имя элемента вроде `product-card__badge-sale`, если по смыслу это тот же бейдж другого типа.

```
<p class="product-card__badge product-card__badge_type_sale">Sale</p>
```

Пример 1 — Бейдж как элемент с модификатором типа

3. Независимость и селекторы

Селектор `.header nav ul li a` зависит от трёх обёрток. Переставили список — стиль умер. Селектор `.menu__link` зависит только от имени. Карточка `.product-card` не должна требовать предка `.products`: её вставляют в `Best sellers`, в модалку и, позже, в корзину.

Связка тега и класса `div.card` повышает специфичность и мешает сменить тег на `article`. Ложный БЭМ — имя с подчёркиваниями при селекторах, которые по-прежнему идут через DOM.

4. Микс, вложенность, файлы

Микс — несколько классов на одном узле: ссылка может быть `menu__link` и `btn`. Блок содержит другие блоки: в `header` лежит `form.search` и `nav.menu`. Запрещён элемент элемента `block__el__sub`: если часть выросла, это новый блок.

Модификатор не переименовывает внутренние элементы. Файл на блок появится полностью на этапе 7; на этапе 3 достаточно дисциплины имён в одном CSS.

5. Практика на CORAL

Было	Стало
header-top	header__top
product-card-media	product-card__media
badge-sale	product-card__badge_type_sale
product-card-selected	product-card_selected

Кнопка btn может стоять на a и на button: БЭМ не отменяет семантику. Длинное имя header-top-nav-item-link-text сигнал, что смешали пять уровней в один класс.

6. Практикумы

6.1. Таблица имён шапки

Выпишите все классы header.

Разметьте блок / элемент / модификатор.

Переименуйте пары HTML+CSS.

Проверьте меню.

6.2. Независимость карточки

Вырежьте article.product-card.

Вставьте его в footer рабочей копии.

Если вид разъехался — найдите селектор с предком.

Верните карточку и исправьте селектор.

7. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Элемент элемента	Имя не читается	Новый блок
Селектор через DOM	Блок не переносится	Один класс
Модификатор слит с элементом	Нельзя сменить тип	Два класса
БЭМ на div вместо article	Теряется семантика	Имя + правильный тег
Два стиля дефисов в одном файле	Команда путается	Только классика курса

8. Контрольные вопросы

1. Что такое блок, элемент и модификатор?
2. Почему рефакторинг делают после простой вёрстки?
3. Чем классика БЭМ отличается от схемы двух дефисов?
4. Почему `.products .item` плох для карточки?
5. Что такое микс?
6. Почему запрещён `block__el__sub`?
7. Как обозначить состояние `selected`?
8. Отменяет ли БЭМ семантические теги?
9. Зачем команде единый словарь имён?
10. Как проверить независимость блока?

9. Итоговая самопроверка

Разберите карточку CORAL: какой класс блок, какие элементы, какие модификаторы `Sale` и `Not`, почему её можно вставить в модалку этапа 4.

Заключение

Этап 3 не меняет картинку. Он меняет договорённость об именах, без которой модалки, JS и адаптив начнут плодить уникальные селекторы. Следующий шаг — слой диалога поверх независимой карточки.

Источники

Каталог `stage-03-bem` репозитория CORAL.
Лекция `docs/course/lectures/03-bem/lecture.md`.
Инструкция `docs/course/guides/03-bem.md`.
`bem.info`: методология и именование.

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

БЭМ · БЛОК · ЭЛЕМЕНТ · МОДИФИКАТОР

Паттерн БЭМ: подробный разбор на проекте CORAL

Дополнение к методическим указаниям этапа 3

Этап 3 — углублённый разбор

Назначение	разобрать методологию БЭМ как рабочий договорённый паттерн, а не набор подчёркиваний
Уровень	базовый / средний
Исходный проект	Coral_Example / stage-02-css → stage-03-bem
Формат работы	чтение имён, таблица переименований, проверка независимости блока

2026

Аннотация

Эти указания разбирают паттерн БЭМ подробнее, чем обзорный текст этапа 3. Материал построен вокруг магазина CORAL: шапка, меню, карточка товара, бейдж Sale, кнопка Shop now и будущее состояние selected, которое повесит JavaScript.

БЭМ здесь понимается как договорённость об именах и границах ответственности. Она не заменяет семантические теги, не рисует сетку и не открывает модальку. Она отвечает на другой вопрос: как назвать кусок интерфейса так, чтобы его можно было перенести, изменить состояние и сопровождать командой.

На курсе принята классическая схема bem.info: block__element и block_modifier, в том числе ключ-значение block_mod_value. Схему двух дефисов (block__element--modifier) знают как соседнюю, но в CORAL не смешивают.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет по любому узлу CORAL сказать, блок это, элемент или модификатор; собрать допустимое имя; объяснить микс; отказаться от элемента элемента; проверить независимость карточки и связать модификатор с будущим classList, не путая его с инлайн-стилем.

Как работать с методическими указаниями

- Держите рядом stage-02-css (простые классы) и stage-03-bem (имена после рефакторинга).
- Каждое правило проверяйте вопросом: «этот селектор выживет, если вырезать HTML и вставить в другое место?»
- Имена произносите вслух: «блок карточка, элемент медиа, модификатор типа sale».
- Не меняйте визуал: критерий понимания — те же пиксели при других именах.
- После теории выполните практикумы по порядку: словарь, карточка, шапка, состояния, ложные имена.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Зачем нужен паттерн, а не «красивые классы»
2. Три сущности: блок, элемент, модификатор
3. Как читается имя
4. Классика БЭМ и схема двух дефисов
5. Принцип независимости
6. Какие селекторы запрещены паттерном
7. Модификатор подробно: булев, ключ-значение, два сразу
8. Микс: несколько блоков на одном узле
9. Почему нет элемента элемента
10. Блок внутри блока
11. БЭМ и семантика HTML

- 12. БЭМ и JavaScript
- 13. Файловая структура как продолжение имён
- 14. Полный разбор карточки CORAL
- 15. Полный разбор шапки CORAL
- 16. Таблица переименований этапа 3
- 17. Практикумы
- 18. Типичные ошибки
- 19. Контрольные вопросы
- 20. Итоговая самопроверка
- Заключение
- Источники

1. Зачем нужен паттерн, а не «красивые классы»

На этапе 2 классы в CORAL простые и понятные: header, hero, product-card, btn. Это нормальная точка старта. Проблема появляется, когда страница растёт. Появляются header-top, product-card-media, badge-sale, selected у карточки в каталоге и у той же карточки в модальке. Селекторы начинают описывать не смысл куска, а случайное дерево: .products .item img, .header nav ul li a.

Паттерн — повторяемое правило, по которому команда принимает одинаковые решения. БЭМ даёт три правила именования и несколько запретов. После этого карточка остаётся карточкой в любом родителе, состояние включается одним классом, а поиск по проекту находит все вхождения product-card__badge.

Без паттерна	С паттерном БЭМ
.products .item .price	.product-card__price
badge-sale как отдельная сущность	product-card__badge_type_sale
style.border из скрипта	product-card_selected
Новые имена на каждую страницу	Один блок, несколько мест

ВАЖНО

БЭМ не делает страницу красивее и не заменяет токены CSS. Если на этапе 2 сетка и цвет собраны плохо, переименование классов это не вылечит. Сначала представление, потом договор об именах.

2. Три сущности: блок, элемент, модификатор

2.1. Блок

Блок — независимый кусок интерфейса, у которого есть смысл даже вне текущего родителя. Его можно перенести, повторить и описать отдельным словарём. Имя блока — существительное или короткое словосочетание через дефис: header, menu, product-card, btn, footer, modal, search-form.

Признак блока: вы можете сказать, зачем он пользователю, не глядя на соседей. Карточка показывает товар. Кнопка запускает действие или переход. Меню даёт набор ссылок каталога. «Обёртка для трёх span» блоком не является, пока у неё нет собственного смысла.

2.2. Элемент

Элемент — часть блока, которая вне этого блока не нужна. Имя всегда содержит имя родителя: product-card__media, product-card__title, product-card__price, header__top, modal__close, search-form__input.

Признак элемента: если вынуть его из блока, имя перестанет быть понятным. Медиа без карточки — просто изображение. Крестик без модальки — просто знак умножения. Поэтому элемент не живёт своей жизнью в CSS как глобальный .media или .close.

2.3. Модификатор

Модификатор уточняет вид, размер, тему или состояние уже существующего блока или элемента. Сам по себе он не существует: нет узла, на котором стоит только `_selected`. Всегда есть базовое имя плюс модификатор: `product-card` `product-card_selected`, `btn` `btn_theme_primary`, `product-card__badge` `product-card__badge_type_sale`.

Признак модификатора: базовый объект остаётся тем же, меняется частность. Карточка остаётся карточкой, но она выбранная. Бейдж остаётся бейджем, но типа `sale`. Если частность настолько другая, что это уже новый смысл, это новый блок, а не модификатор.

Сущность	Вопрос	Пример CORAL
Блок	Что это за кусок UI?	<code>product-card</code> , <code>header</code> , <code>btn</code>
Элемент	Какая это часть блока?	<code>product-card__media</code>
Модификатор	Какой это вариант?	<code>product-card_selected</code>

3. Как читается имя

Имя БЭМ читают слева направо. Сначала сущность, потом уточнение. Дефис внутри слова — часть одного имени: `product-card` это один блок, не `product` плюс `card`. Два подчёркивания отделяют элемент. Одно подчёркивание отделяет модификатор и, если нужно, его значение.

<code>product-card</code>	→ блок «карточка товара»
<code>product-card__media</code>	→ элемент «медиа» блока «карточка»
<code>product-card__badge</code>	→ элемент «бейдж»
<code>product-card__badge_type_sale</code>	→ тот же бейдж, тип <code>sale</code>
<code>product-card_selected</code>	→ блок карточки в состоянии <code>selected</code>
<code>btn_theme_primary</code>	→ блок кнопки, тема <code>primary</code>

Пример 1 — Как произносить имя

Полезно завести привычку не смотреть на подчёркивания как на украшение. Каждое `_` и `--` несёт синтаксис. Если в имени четыре дефиса и два подчёркивания подряд без правила, это уже не паттерн, а длинная подпись.

4. Классика БЭМ и схема двух дефисов

В индустрии живут две массовые записи. Классика Яндекса: элемент через `_`, модификатор через одно подчёркивание, значение через ещё одно. Схема, которую часто называют Two Dashes или SUIT-подобной: элемент через `_`, модификатор через два дефиса.

Смысл	Классика курса	Схема двух дефисов
Элемент медиа	<code>product-card__media</code>	<code>product-card__media</code>
Состояние <code>selected</code>	<code>product-card_selected</code>	<code>product-card--selected</code>
Тип бейджа <code>sale</code>	<code>product-card__badge_type_sale</code>	<code>product-card__badge--sale</code>
Тема кнопки	<code>btn_theme_primary</code>	<code>btn--primary</code>

Обе записи валидны, если файл и команда держатся одной. На курсе CORAL выбрана классика, потому что так написана документация bem.info и так уже названы классы эталона stage-03-bem. Смешивать product-card--selected и header__top_sticky в одном проекте нельзя: поиск и договорённость рассыпаются.

ВАЖНО

Не «исправляйте» эталон курса на два дефиса из внешней статьи. Паттерн работает, только когда он общий.

5. Принцип независимости

Главный практический смысл БЭМ — независимость блока от места в дереве. Карточка в каталоге Products, карточка в Best sellers и карточка внутри модальки товара — один блок product-card. Если стиль цены описан как .products .item span, во втором месте цена «потеряет одежду».

Независимость не значит, что блок нельзя класть внутрь другого блока. Значит другое: внешний блок не должен навязывать внутреннему внешность через вложенный селектор. Header может содержать menu, но цвет ссылки задаёт .menu__link, а не .header nav a.

1. Вырезать `<article class="product-card">...</article>`.
2. Вставить в footer или в .modal_dialog.
3. Если фото, название, цена и бейдж выглядят так же — блок независим.
4. Если что-то «побледнело», ищите селектор с предком.

Пример 2 — Проверка независимости

Исключение, которое всё же допустимо — модификатор внешнего блока, влияющий на внутренний по явному правилу проекта, например .modal_open .modal_dialog. Это состояние самого модального блока, а не тайная зависимость карточки от каталога.

6. Какие селекторы запрещены паттерном

БЭМ в CSS почти всегда — один класс. Не потому, что другие селекторы «незаконны в языке», а потому что они ломают независимость и предсказуемую специфичность.

Селектор	Почему паттерн его не берёт	Чем заменить
.header nav ul li a	Зависит от трёх обёрток	.menu__link
div.product-card	Нельзя сменить div на article	.product-card
#products .card	id уникален, блок не повторить	.product-card
.products .price	Цена привязана к секции	.product-card__price
article > h3	Тег вместо роли	.product-card__title

Один класс даёт одинаковый вес. Тогда модификатор, записанный рядом, уверенно перекрывает базу: `.product-card_selected` сильнее не из-за `!important`, а потому что он такой же по специфичности и стоит ниже или описывает более частный случай тем же классом.

Комбинация двух классов на одном узле допустима и как раз является штатным приёмом: `.product-card.product-card_selected`. Это не «селектор через DOM», это узел, который одновременно база и состояние.

7. Модификатор подробно: булев, ключ-значение, два сразу

7.1. Булев модификатор

Булев модификатор включает признак, у которого нет шкалы значений. Карточка либо выбранная, либо нет. Меню либо открыто, либо нет. Модалка либо открыта, либо нет. Запись: `product-card_selected`, `menu_open`, `modal_open`.

7.2. Ключ-значение

Когда у признака есть несколько взаимоисключающих вариантов, пишут ключ и значение: `product-card__badge_type_sale` и `product-card__badge_type_hot`, `btn_size_s` и `btn_size_l`, `btn_theme_primary` и `btn_theme_line`. Так в поиске видно, что `type` — одна ось, а `sale` и `hot` — точки на ней.

```
<p class="product-card__badge product-card__badge_type_sale">Sale</p>
<p class="product-card__badge product-card__badge_type_hot">Hot</p>
```

Пример 3 — Два класса: база элемента и модификатор

Не сливайте это в `product-card__badge-sale`. Тогда `sale` перестаёт быть вариантом бейджа и становится новым элементом. Добавить `Hot` придётся третьим именем, и общее оформление бейджа придётся копировать.

7.3. Несколько модификаторов

На одном узле могут стоять несколько модификаторов разных осей: размер и тема, выбранность и скидка. Нельзя ставить два значения одной оси сразу: бейдж не бывает `type_sale` и `type_hot` одновременно, если по смыслу это один ярлык.

```
<article class="product-card product-card_selected">
  <p class="product-card__badge product-card__badge_type_sale">Sale</p>
</article>
```

Пример 4 — Разные оси совместимы

8. Микс: несколько блоков на одном узле

Микс — когда один HTML-узел одновременно играет роль двух блоков или блока и элемента другого блока. Это не нарушение, а штатный приём. Ссылка в меню может выглядеть как кнопка: `class="menu__link btn btn_theme_primary"`. Узел остаётся ссылкой по тегу `a` и получает внешность кнопки по блоку `btn`.

В CORAL микс появляется там, где семантика и внешность расходятся. `Shop now` — ссылка на `#products`, но выглядит как кнопка. `View product` — тоже ссылка. `Search` — настоящая `button` `type="submit"`. Внешность общая, тег разный. БЭМ как раз позволяет повесить `btn` на оба тега.

```
<a class="btn btn_theme_primary" href="#products">Shop now</a>
<button class="btn btn_theme_primary" type="submit">Sign in</button>
```

Пример 5 — Микс ссылки и кнопки

Правило микса: каждый класс отвечает за своё. `btn` описывает `padding`, фон, кегль. `menu__link` описывает роль в меню, если она нужна сверх кнопки. Не запикивайте навигационный отступ в `btn_theme_primary`: тема кнопки начнёт ломать кнопку в форме.

9. Почему нет элемента элемента

Запись `block__el__sub` запрещена не из вкуса, а потому что ломает границу блока. Если у карточки появляется `product-card__media__caption`, подпись уже достаточно самостоятельна, чтобы стать либо элементом карточки `product-card__caption`, либо новым блоком `image-caption` внутри медиа.

Цепочка `__` создаёт ложную вложенность имён: перенесли подпись — имя всё ещё требует `media`. Паттерн требует плоского списка элементов блока. Вложенность HTML при этом может быть любой глубины.

Плохо	Хорошо
<code>product-card__media__caption</code>	<code>product-card__caption</code>
<code>header__top__search__input</code>	<code>search-form__input</code> внутри блока <code>search-form</code>
<code>footer__nav__item__link</code>	<code>menu__link</code> внутри блока <code>menu</code>

ПРАВИЛО

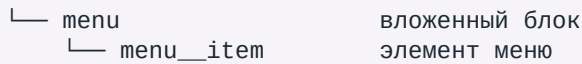
HTML можно вкладывать глубоко. Имена БЭМ — нет. Если хочется третьего `__`, выделите новый блок.

10. Блок внутри блока

Страница CORAL — дерево блоков, а не один гигантский `page__....` Header содержит `search-form`, `logo` и `menu`. Catalog содержит `product-card`. Modal содержит `product-card` или формы. Внешний блок не переименовывает внутренний: внутри header карточки нет `header__product-card`.

Когда внешний блок должен чуть сдвинуть внутренний, используют микс на внутреннем узле или элемент-обёртку. Обёртка `header__account` — элемент шапки, внутри него уже ссылки или иконки. Сама иконка может быть отдельным маленьким блоком `icon`, если её переиспользуют в подвале и в кнопке поиска.

header	блок
├ header__top	элемент-раскладка
│ └ search-form	вложенный блок
│ └ logo	вложенный блок
└ header__user	элемент



Пример 6 — Шапка как контейнер блоков

11. БЭМ и семантика HTML

Паттерн имён не отменяет выбор тега. `article` остаётся `article`, `nav` остаётся `nav`, `button` остаётся `button`. Класс говорит «какой это блок», тег говорит «какой это смысл для браузера и вспомогательных технологий».

Узел CORAL	Тег	Классы
Карточка	<code>article</code>	<code>product-card</code>
Название в карточке	<code>h3</code>	<code>product-card__title</code>
Меню каталога	<code>nav</code>	<code>menu</code>
Shop now	<code>a</code>	<code>btn btn_theme_primary</code>
Отправка входа	<code>button</code>	<code>btn btn_theme_primary</code>

Ошибка «сделать всё `div`, зато с правильным БЭМ» возвращает проект к этапу до семантики. Ошибка «оставить `article`, но селектор `article h3`» возвращает к этапу до независимости. Нужны оба слоя сразу.

12. БЭМ и JavaScript

Скрипт не должен становиться таблицей стилей. Выбранная карточка не получает `element.style.outline`. Она получает модификатор `product-card_selected`. Открытая модалка — `modal_open`. Открытое меню на 768 px — `menu_open` или `header_menu-open`, по словарю проекта.

```

card.classList.add("product-card_selected");
modal.classList.add("modal_open");
menu.classList.toggle("menu_open");
  
```

Пример 7 — Состояние включает класс, не инлайн

Для поиска узла скриптом иногда добавляют отдельный JS-класс по договорённости, например `js-product-card`. На курсе CORAL чаще используют уже существующие БЭМ-имена и `data`-атрибуты: `data-catalog`, `data-product-id`. `data-*` не заменяет модификатор внешности: атрибут хранит факт, класс рисует состояние.

Имя модификатора должно совпадать в CSS и в JS буквально. Опечатка `product-card_selecte` оставит рамку мёртвой. Поэтому словарь состояний лучше выписать до кода: `selected`, `open`, `disabled`, `current`.

13. Файловая структура как продолжение имён

На этапе 3 ещё можно держать один CSS-файл, если имена уже бэмовские. На этапе 7 структура повторяет словарь блоков: `css/blocks/product-card/product-card.css`,

css/blocks/header/header.css. Один блок — один файл. Модификаторы блока живут в том же файле, пока их немного.

Так паттерн замыкается: имя в HTML, правило в CSS, папка на диске и класс в JS говорят об одной сущности. Искать product-card__badge проще, чем вспоминать, в каком хвосте style.css лежала скидка.

14. Полный разбор карточки CORAL

Карточка — лучший учебный объект, потому что в ней есть все три сущности, повтор, состояние и будущий перенос в модалку.

```
<li>
  <article class="product-card">
    <a class="product-card__media" href="product.html">
      
      <h3 class="product-card__title">Nike Sportswear Futura Luxe</h3>
    </a>
    <p class="product-card__category">Bag</p>
    <p class="product-card__price"><data value="130.00">$130.00</data></p>
    <p class="product-card__badge product-card__badge_type_sale">Sale</p>
    <a class="product-card__action btn" href="product.html">View product</a>
  </article>
</li>
```

Пример 8 — Карточка после рефакторинга

Класс	Роль в паттерне	Почему так
product-card	блок	Самостоятельная единица витрины
product-card__media	элемент	Кликабельная связка фото и названия
product-card__image	элемент	Само фото; вне карточки не нужно
product-card__title	элемент	Заголовок именно карточки
product-card__category	элемент	Тип товара
product-card__price	элемент	Цена; data остаётся семантикой
product-card__badge	элемент	Ярлык на карточке
product-card__badge_type_sale	модификатор элемента	Вариант ярлыка
product-card__action	элемент	Явное действие View product
btn	микс блока	Внешность кнопки на ссылке
li	не БЭМ	Принадлежность к списку HTML

Элемент li не получает класс product-list__item обязательно, но это хороший кандидат, если у пункта списка есть своя раскладка. Саму карточку вешать классом на li не стоит: тогда article потеряет имя блока, а список начнёт владеть карточкой.

15. Полный разбор шапки CORAL

Шапка показывает другую сторону паттерна: крупный блок-контейнер и вложенные самостоятельные блоки. Не всё внутри header обязано называться header__....

Кусок шапки	Имя	Почему не элемент шапки
Верхний ряд	header__top	Это раскладка именно шапки
Поиск	search-form	Форма переиспользуема, у неё свои поля
Поле поиска	search-form__input	Часть формы, не шапки
Логотип	logo	Ведёт на главную и с других страниц
Account / корзина	header__user	Группа именно шапки
Каталожное меню	menu	То же меню по смыслу может быть в подвале
Пункт меню	menu__item	Часть меню
Ссылка пункта	menu__link	Часть меню

Если назвали поиск header__search__input, вы уже построили элемент элемента и одновременно запретили перенести форму. Правильная граница: шапка владеет местом, форма владеет полями.

16. Таблица переименований этапа 3

Рефакторинг начинают со словаря, не с массовой замены вслепую. Ниже — ядро CORAL. Полный список студент дополняет по своему файлу этапа 2.

Было на этапе 2	Стало в БЭМ	Комментарий
header-top	header__top	элемент шапки
search-form	search-form	уже блок, можно не трогать
product-card	product-card	уже блок
product-card-media	product-card__media	элемент
product-card-title	product-card__title	элемент
badge-sale	product-card__badge_type_sale	элемент + модификатор
badge-hot	product-card__badge_type_hot	та же ось type
product-card-selected	product-card_selected	состояние блока
btn-primary	btn_theme_primary	ось theme
hero-content	hero__content	элемент героя
footer-nav	меню как блок menu внутри footer	не footer__nav__link

Правило замены: HTML и CSS правят парами. Переименовали класс в разметке — сразу селектор. Иначе полчаса страница живёт без стилей, и студент начинает «чинить» это инлайном.

17. Практикумы

17.1. Словарь одного экрана

Откройте index.html этапа 2 и выпишите все class шапки и одной карточки.
Пометьте каждую запись: блок / элемент / модификатор / пока неясно.
Для неясных имён задайте вопрос: это может жить в другом родителе?
Соберите таблицу «было → стало» на 12+ строк.

17.2. Независимость карточки

В рабочей копии вырежьте одну article.product-card.
Вставьте её первым ребёнком footer.
Сверьте фото, заголовок, цену, бейдж и кнопку.
Любое расхождение превратите в селектор с предком и перепишите на класс блока.

17.3. Ось модификатора бейджа

Найдите Sale и Hot.
Сделайте общий элемент product-card__badge.
Добавьте type_sale и type_hot.
Объясните, почему не нужны два элемента badge-sale и badge-hot.

17.4. Микс кнопки

Найдите Shop now, View product, Search и Sign in.
Разделите тег и внешность.
Повесьте общий блок btn там, где внешность общая.
Оставьте отличия темы или размера модификаторами, а не новыми блоками.

17.5. Запрет элемента элемента

Найдите в своей черновой копии любое имя с двумя __.
Решите: это элемент исходного блока или новый блок?
Переименуйте и поправьте CSS.
Запишите правило своими словами.

17.6. Состояние для будущего JS

Опишите CSS для product-card_selected: рамка, тень или фон.
Вручную повесьте класс на одну карточку в HTML.
Снимите класс — вид должен вернуться.
Объясните, почему это готовый рычаг для classList на этапе 5.

18. Типичные ошибки

Ошибка	Почему это ломает паттерн	Как исправить
Подчёркивания без границ сущностей	Имя нельзя произнести по правилу	Сначала блок, потом __ или _
block__el__sub	Ложная вложенность имён	Плоский элемент или новый блок
.block .block__el	Лишний предок, выше специфичность	Только .block__el
div.product-card	Тег приклеен к имени	Один класс
badge-sale как элемент	Нет оси вариантов	badge + type_sale
Один модификатор без базы	Нечего модифицировать	Всегда два класса
btn_theme_primary лезет в отступ меню	Микс начинает врать	Отступ — элемент меню
Инлайн из JS вместо _selected	Состояние не в словаре	classList и CSS
Два стиля дефисов в одном файле	Команда теряет поиск	Только классика курса
Все классы header__...	Шапка поглощает чужие блоки	Вложенные logo, menu, search-form
БЭМ на сплошных div	Потеряна семантика	Тег по смыслу, класс по блоку
Утилиты m-24 вместо элемента	Ритм размазан по узлам	Отступ блока или gap родителя

18.1. Чек-лист имени

Имя произносится как блок, элемент или модификатор.

Нет второго __.

Модификатор стоит вместе с базой, не вместо неё.

Селектор CSS — класс без цепочки тегов.

Блок можно перенести в другой родитель.

Состояние, которое включит JS, уже существует как модификатор.

Тег выбран по смыслу, не по удобству селектора.

19. Контрольные вопросы

1. Какой вопрос отличает блок от элемента?
2. Почему модификатор не существует без базового класса?
3. Как в классике БЭМ записывают элемент и модификатор?
4. Чем запись product-card_selected отличается от product-card--selected и почему на курсе выбрана первая?
5. Что проверяет перенос карточки в подвал?
6. Почему селектор .header nav ul li a нарушает паттерн?

7. Зачем оси type у бейджа, а не два элемента sale и hot?
8. Можно ли повесить два модификатора на один узел? Когда нельзя?
9. Что такое микс и зачем он Shop now?
10. Почему запрещён product-card__media__caption?
11. Почему меню в шапке — отдельный блок, а не header__menu__link?
12. Отменяет ли БЭМ теги article, nav и button?
13. Как JavaScript должен включать выбранную карточку?
14. Чем data-product-id отличается от модификатора selected?
15. Почему рефакторинг начинают с таблицы имён, а не с Replace All?
16. Что остаётся не-бэмовским в разметке карточки и почему это нормально?
17. Как файл product-card.css на этапе 7 продолжает паттерн этапа 3?
18. Почему утилита margin-24 опасна как замена элемента?
19. Какая ошибка делает «ложный БЭМ» — подчёркивания при селекторах через DOM?
20. Как понять, что блок описан достаточно, чтобы перенести его в модалку этапа 4?

20. Итоговая самопроверка

Возьмите одну карточку и шапку CORAL. Для каждого класса заполните строку: сущность паттерна, зачем пользователю, можно ли перенести, кто включит состояние, какой тег рядом. Если строка не собирается, имя ещё не стало паттерном.

Шаг	Что объяснить
1. Граница блока	Где кончается product-card и начинается список или секция
2. Список элементов	Почему media, title, price — части карточки
3. Оси модификаторов	selected у блока, type у бейджа
4. Микс	Где btn накладывается на ссылку
5. Селекторы	Почему в CSS нет предка .products
6. JS-рычаг	Какой класс добавит этап 5
7. Перенос	Что произойдёт в модалке и в подвале

КРИТЕРИЙ ПОНИМАНИЯ

Студент не пересказывает «блок-элемент-модификатор» как слоган, а на живом узле CORAL объясняет границу ответственности имени, селектора и тега.

Заключение

Паттерн БЭМ в CORAL — это словарь границ. Блок можно перенести. Элемент живёт только в блоке. Модификатор включает вариант, не создавая новую сущность. Микс разрешает одному узлу быть и ссылкой, и кнопкой внешне. Запрет элемента элемента не даёт именам повторять случайную вложенность HTML.

Когда словарь собран, модалка этапа 4 получает готовую карточку, скрипт этапа 5 получает готовые рычаги `_open` и `_selected`, адаптив этапа 7 получает папки блоков. Без этого паттерн остаётся декоративными подчёркиваниями.

Источники

БЭМ. Методология и соглашение об именовании. <https://ru.bem.info/methodology/> (дата обращения: 15.09.2026).

Ключевые понятия: блок, элемент, модификатор. <https://ru.bem.info/methodology/key-concepts/> (дата обращения: 15.09.2026).

Учебный эталон CORAL, каталог stage-03-bem. https://github.com/kruzenshteyn/Coral_Example (дата обращения: 15.09.2026).

Лекция этапа 3. [docs/course/lectures/03-bem/lecture.md](https://docs.course/lectures/03-bem/lecture.md).

Инструкция этапа 3. docs/course/guides/03-bem.md.

MDN. Атрибут `class` и специфичность селекторов. <https://developer.mozilla.org/ru/> (дата обращения: 15.09.2026).

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

DIALOG · OVERLAY · :TARGET

Модальные окна входа и карточки товара

Методические указания на примере проекта CORAL

Этап 4 — Модалки

Назначение	вёрстка диалогов без JavaScript, с заделом под этап 5
Уровень	базовый
Исходный проект	Coral_Example / stage-04-modals
Формат работы	разметка dialog, оверлей, :target, фокус, формы внутри

2026

Аннотация

Методические указания посвящены модальным окнам интернет-магазина CORAL. В макете отдельного фрейма авторизации нет: диалоги собирают по токенам Primary #9E5CF2, типографике Roboto / Open Sans и кнопкам shop now. В каталоге stage-04-modals два окна: #modal-auth и #modal-product.

На этапе 4 окна открываются через :target. Класс modal_open уже описан в CSS и понадобится скрипту этапа 5. Издание разбирает оверлей, position: fixed, роль dialog, имя через aria-labelledby, закрытие по ссылке на # и границу возможностей HTML+CSS.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет собрать слой диалога, открыть его ссылкой Account без JavaScript, связать заголовок с aria-labelledby, не смешивать hidden с классом открытия и объяснить, какие операции с фокусом ещё предстоят.

Как работать с методическими указаниями

- Откройте stage-04-modals/index.html и css/modal.css.
- Кликните Account и карточку товара: должен смениться hash.
- Проверьте Tab до крестика и полей.
- Не подключайте скрипт открытия — это следующий этап.
- Сверяйте цвет и типографику с токенами CORAL, а не с чужим UI-китом.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа
2. Слой поверх страницы
3. Доступность диалога
4. Открытие через :target
5. Два окна CORAL
6. Практикумы
7. Типичные ошибки
8. Контрольные вопросы
9. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

Попап — отдельный слой интерфейса: оверлей, фокус, смысл «пока открыт диалог, страница не обычный контент». Сначала вёрстка, потом поведение. Нативный элемент `dialog` в проде уместен; на курсе слой собирают руками, чтобы были видны `z-index`, оверлей и класс `_open`.

ВАЖНО

ARIA описывает интерфейс, но не открывает окно. Неработающий элемент с правильной ролью остаётся неработающим.

2. Слой поверх страницы

Оверлей затемняет фон и ловит клик «заккрыть». Диалог позиционируют `position: fixed` относительно окна, не документа. `z-index` должен быть выше шапки и баннера. `alert()` не заменяет этот слой: его нельзя сверстать токенами CORAL и вложить в него форму.

3. Доступность диалога

`role="dialog"`, `aria-modal="true"` и `aria-labelledby` по `id` заголовка задают тип и имя. Крестик — ссылка или `button` с `aria-label Close`, не пустой `span`. Фокус внутрь, ловушка `Tab`, `inert` на фоне и возврат фокуса на вызывающий элемент появятся на этапе 5. На этапе 4 проверяют, что эти узлы вообще есть в разметке.

4. Открытие через `:target`

Ссылка `href="#modal-auth"` ставит идентификатор в адрес. Псевдокласс `:target` показывает соответствующий `.modal`. Закрытие — ссылка на `#`. Атрибут `hidden` в UA-стилях даёт `display: none` сильнее обычного класса, поэтому на этом этапе сккрытие держат в CSS: `visibility` и `pointer-events`.

```
.modal_open,
.modal:target {
  visibility: visible;
  pointer-events: auto;
}
```

Пример 1 — Открытое состояние

5. Два окна CORAL

Окно	Зачем	Как открыть без JS
<code>#modal-auth</code>	Вход и регистрация	Account → <code>#modal-auth</code>
<code>#modal-product</code>	Карточка товара	Карточка → <code>#modal-product</code>

Внутри `auth` две формы с собственными полями, как на `login.html`. Внутри `product` — изображение, цена, описание и кнопка `Select`. Два диалога сразу не открывают: один слой — одно модальное состояние.

6. Практикумы

Соберите разметку `#modal-auth` с оверлеем, крестиком и `h2`.

Свяжите `Account` с `href="#modal-auth"`.

Откройте окно без скрипта и закройте ссылкой на `#`.

Проверьте, что поля имеют `label`.

Повторите для `#modal-product`.

7. Типичные ошибки

Ошибка	Почему это проблема	Исправление
hidden + класс открытия	<code>hidden</code> побеждает CSS	Один механизм скрытия
Крестик-span	Нет в порядке Tab	Ссылка или <code>button</code>
alert() вместо слоя	Нет токенов и формы	Оверлей + <code>dialog</code>
Два окна :target сразу	Неясный фокус	Одно открытое состояние
Нет имени диалога	Скринридер молчит	<code>aria-labelledby</code>

8. Контрольные вопросы

1. Чем диалог отличается от обычной секции?
2. Зачем оверлей?
3. Почему `position: fixed`, а не `relative`?
4. Что даёт `:target`?
5. Почему `hidden` мешает классу открытия?
6. Какие ARIA-атрибуты уже ставят на этапе 4?
7. Какие операции с фокусом откладывают на JS?
8. Почему в проекте есть и страницы, и модалки?
9. Чем крестик отличается от `submit` формы?
10. Почему на курсе не начинают с `dialog.showModal()`?

9. Итоговая самопроверка

Разберите `#modal-auth` по шаблону: назначение, границы, имя, вложенность, навигация по `hash`, что уже работает в HTML/CSS и что потребует JS.

Заключение

Этап 4 закрывает пробел макета: окон входа на холсте не было, но дизайн-система позволяет их собрать. Следующий шаг — переключать класс `modal_open` скриптом и провести фокус.

Источники

Каталог stage-04-modals.

Лекция docs/course/lectures/04-modals/lecture.md.

MDN: :target, role dialog, aria-modal, элемент dialog.

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

JS · DOM · FETCH

Поведение витрины, модалок и форм на JavaScript

Методические указания на примере проекта CORAL

Этап 5 — JavaScript

Назначение	оживить каталог, диалоги, валидацию и выбор товара
Уровень	базовый
Исходный проект	Coral_Example / stage-05-js
Формат работы	локальный сервер, fetch JSON, classList, localStorage

2026

Аннотация

Методические указания посвящены клиентскому JavaScript магазина CORAL. Карточки больше не зашиты в HTML: их рисует `js/products.js` из `data/products.json`. Файл `js/main.js` открывает модальки классом `modal_open`, проверяет формы, фильтрует Best sellers, включает бургер и помнит выбор в `localStorage`.

`fetch` не работает с `file://`. Этап запускают командой `python -m http.server 8080`. Издание разбирает привязки языка, DOM и делегирование, асинхронность, данные и границу: клиентская проверка удобна, но не заменяет сервер.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет поднять локальный сервер, загрузить каталог через `fetch`, отрисовать карточки без `innerHTML` из JSON, открыть и закрыть диалог классом, не отправить невалидный email и объяснить сценарий `Select product` без входа.

Как работать с методическими указаниями

- Запускайте страницу только через HTTP, не как файл с диска.
- Сначала добейтесь сетки из JSON, затем модальки, затем валидацию.
- Текст товара вставляйте через `textContent`.
- Слушатель клика вешайте на родителя, а не на каждую будущую карточку вслепую.
- Помните: `localStorage` — учебное хранилище того же origin, не сессия сервера.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа

2. Язык и данные

3. DOM и события

4. Сеть и асупс

5. Сценарии CORAL

6. Практикумы

7. Типичные ошибки

8. Контрольные вопросы

9. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

Страница оживает: данные не защиты в разметку, диалоги открываются классом, форма проверяется до отправки. Стили остаются в CSS: скрипт не становится «стилистом» через `element.style`.

1.2. Состав каталога

Файл	Роль
<code>data/products.json</code>	Каталог: id, title, price, image, бейджи
<code>js/products.js</code>	fetch и отрисовка сеток
<code>js/main.js</code>	модалки, валидация, выбор, фильтр, бургер

2. Язык и данные

`const` запрещает переназначить привязку, не делает объект неизменным. `let` ограничен блоком, `var` — нет. Сравнение берут `===`. `typeof null` даёт `object` — это историческая ловушка. Модули и «use strict» отключают случайные глобалы.

`JSON.parse` бросает исключение на битой строке. Массивы каталога фильтруют и отображают через `filter` и `map`. `localStorage` хранит строки того же `origin`: объекты кладут через `JSON.stringify`.

3. DOM и события

`querySelector` возвращает первый узел или `null` — обращение к свойству без проверки роняет скрипт. `addEventListener` не затирает чужие обработчики, в отличие от `onclick`-свойства. `preventDefault` на `submit` останавливает переход `action="#"`.

Делегирование: один слушатель на `document` ловит клик по карточке, которой ещё не было в DOM в момент загрузки. `closest(".product-card")` находит карточку от цели события. `classList.toggle` и `dataset.productId` связывают узел с `id` товара.

```
document.addEventListener("click", (event) => {
  const media = event.target.closest(".product-card__media");
  if (!media) return;
  // открыть модалку товара
});
```

Пример 1 — Делегирование клика по карточке

4. Сеть и async

`fetch` не бросает исключение на 404: проверяют `response.ok`. `async`-функция всегда возвращает `Promise`. `await` допустим внутри `async` или на верхнем уровне модуля. `type="module"` даёт строгий режим и отложенную загрузку.

```
async function loadProducts() {
```

```
const response = await fetch("data/products.json");
if (!response.ok) throw new Error("HTTP " + response.status);
return response.json();
}
```

Пример 2 — Загрузка каталога

5. Сценарии CORAL

Карточка → модалка товара → Select product без входа → модалка Account.

Регистрация или вход → Select product → бейдж корзины, рамка product-card_selected, значение в localStorage.

Escape и клик по оверлею закрывают диалог.

Невалидный email не отправляет форму.

ВАЖНО

Клиентская валидация повышает удобство и не является механизмом безопасности. Сервер этапа 6 обязан проверить значения снова.

6. Практикумы

6.1. Каталог из JSON

Поднимите python -m http.server 8080.

Загрузите products.json.

Нарисуйте карточки в [data-catalog] через createElement.

Откройте file:// и объясните ошибку.

6.2. Модалка и выбор

Переключайте modal_open.

Закройте окно по Esc и оверлею.

Свяжите Select с проверкой «есть ли пользователь».

Обновите страницу и убедитесь, что выбор жив.

7. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Открыть HTML с диска	fetch блокируется	Локальный HTTP
innerHTML из JSON	XSS из названия	textContent
style.display в скрипте	Стили размазаны	classList
Слушатель в цикле до DOM	Карточек ещё нет	Делегирование
Не проверить response.ok	Тихий пустой каталог	Бросить ошибку

8. Контрольные вопросы

1. Чем `const` отличается от `let` и `var`?
2. Почему сравнивают через `===`?
3. Что возвращает `querySelector`, если узла нет?
4. Зачем `preventDefault` на `submit`?
5. Что такое делегирование событий?
6. Почему `fetch` не падает на 404?
7. Почему `file://` ломает загрузку JSON?
8. Чем `textContent` безопаснее `innerHTML`?
9. Что хранит `localStorage` и чего он не заменяет?
10. Какой сценарий связывает `Select product` и `Account`?

9. Итоговая самопроверка

Разберите путь клика по карточке: событие, `closest`, поиск товара в массиве, заполнение модальки, условие входа, запись в `localStorage`.

Заключение

Этап 5 делает витрину живой, но данные всё ещё живут в браузере. Следующий шаг — один HTTP-контракт на трёх серверах.

Источники

Каталог `stage-05-js`.

Лекция `docs/course/lectures/05-js/lecture.md`.

MDN: DOM, события, `fetch`, `localStorage`, Form validation.

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

HTTP · API · СЕССИЯ

Серверный API интернет-магазина на трёх стеках

Методические указания на примере проекта CORAL

Этап 6 — Сервер

Назначение	один контракт API в PHP, Django и Node.js
Уровень	базовый / средний
Исходный проект	Coral_Example / stage-06-server
Формат работы	контракт, curl, cookie-сессия, общий фронт

2026

Аннотация

Методические указания посвящены появлению HTTP в проекте CORAL. Браузер не хранит пароли надёжно и не отдаёт общий каталог всем устройствам. Появляются метод, путь, статус, JSON и cookie-сессия. Один фронт frontend/ и три бэкенда разделяют api-contract.md.

Формат ответа единый: { ok, data } или { ok, error }. Порты занятия: PHP 8000, Django 8001, Node.js 8002. Статику отдают с того же origin, чтобы не решать CORS раньше времени.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет прочитать контракт, поднять один стек, получить каталог curl-запросом, зарегистрировать пользователя, отличить 401 от 409 и объяснить, почему пароль не возвращают в JSON.

Как работать с методическими указаниями

- Сначала читайте api-contract.md, потом код стека.
- Проверьте API curl или аналогом до открытия браузера.
- Не запускайте Live Server на одном порту и API на другом без необходимости.
- Сравнивайте тела ответов трёх стеков по смыслу, не по пробелам.
- Сессию смотрите в Set-Cookie, не в localStorage.

Оглавление

[Аннотация](#)

[Как работать с методическими указаниями](#)

- [1. Назначение этапа](#)
- [2. HTTP: метод, статус, JSON](#)
- [3. Сессия и безопасность](#)
- [4. Три стека, один контракт](#)
- [5. Практикумы](#)
- [6. Типичные ошибки](#)
- [7. Контрольные вопросы](#)
- [8. Итоговая самопроверка](#)

[Заключение](#)

[Источники](#)

1. Назначение этапа

Протокол важнее синтаксиса языка. PHP показывают первым как самый прямой HTTP, затем Django и Node. Фреймворки нарочно тонкие: без Laravel и без DRF, чтобы были видны путь, заголовок и статус.

2. HTTP: метод, статус, JSON

GET читает каталог и текущего пользователя. POST создаёт регистрацию, вход и выбор товара. Коды не декоративны: 401 — кто вы; 403 — вам нельзя; 404 — ресурса нет; 409 — email уже занят; 400 — тело неверно.

```
{ "ok": true, "data": { "id": 1, "name": "Ada", "email": "ada@example.com" } }
{ "ok": false, "error": "Email already registered" }
```

Пример 1 — Единая оболочка ответа

Фронт смотрит поле ok. Код ответа всё равно ставят правильный: так работают мониторинг и curl.

3. Сессия и безопасность

Сессию кладут в cookie. Флаг HttpOnly закрывает её от чужого скрипта страницы лучше, чем JWT в localStorage на этом этапе обучения. Пароль хранят через password_hash стека, в JSON пользователя поля password нет.

CSRF в учебном JSON-API на занятии могут отключить, но тема остаётся в вопросах теста: cookie уходит на origin сама, поэтому изменяющие методы в проде защищают токеном или тем же сайтом.

4. Три стека, один контракт

Стек	Запуск	Порт
PHP	php -S localhost:8000 router.php из php/	8000
Django	python manage.py runserver 8001 из django/	8001
Node.js	npm install && npm start из nodejs/	8002

GET /api/products не содержит паролей. GET /api/me без cookie даёт 401, с cookie — текущего пользователя. Выбор товара после входа переживает обновление страницы, потому что живёт уже не только в localStorage.

5. Практикумы

Прочитайте таблицу путей контракта.

Поднимите PHP и запросите /api/products.

Зарегистрируйте пользователя из файла JSON.

Вызовите /api/me с cookie и без.
Повторите GET каталога на втором стеке.

6. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Разный JSON у стеков	Фронт ветвится	Общий контракт
Пароль в ответе	Утечка	Только hash в хранилище
Токен в localStorage «по привычке»	XSS читает его	Учебная cookie + HttpOnly
Два origin без CORS	Браузер режет fetch	Статика с того же сервера
200 на занятый email	Клиент не видит конфликт	409 и ok: false

7. Контрольные вопросы

1. Чем GET отличается от POST?
2. Что означает 401, 403, 404 и 409?
3. Зачем единое поле ok?
4. Почему три языка и один контракт?
5. Куда кладут сессию на этом этапе?
6. Почему пароль не возвращают клиенту?
7. Зачем отдавать HTML с того же origin, что и API?
8. Что проверяет TESTING.md?
9. Чем серверная проверка лучше required в форме?
10. Почему PHP показывают раньше фреймворков?

8. Итоговая самопроверка

Опишите регистрацию: метод, путь, тело, успешный ответ, ответ при занятом email, появление cookie, последующий GET /api/me.

Заключение

Этап 6 переносит истину данных на сервер. Хранилище пока файловое или простое. Следующие шаги — адаптив фронта и замена хранилища на SQLite при том же контракте.

Источники

stage-06-server/api-contract.md и TESTING.md.
Лекция docs/course/lectures/06-server/lecture.md.
MDN: HTTP, cookie, CORS, коды ответа.

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

ADAPTIVE · @MEDIA · БЭМ-ПАПКИ

Адаптив и CSS, разложенный по блокам

Методические указания на примере проекта CORAL

Этап 7 — Адаптив

Назначение	desktop-first адаптив и файловая структура БЭМ
Уровень	базовый / средний
Исходный проект	Coral_Example / stage-07-adaptive
Формат работы	папки блоков, @media внутри блока, проверка 1280/768/375

2026

Аннотация

Методические указания посвящены двум задачам сразу: магазин должен читаться на 375 px, а CSS перестаёт быть монолитом. Каждый блок живёт в `css/blocks/<имя>/<имя>.css`, адаптив пишут рядом с базой блока, а не свалкой в конце `style.css`.

Макет Figma — desktop. Курс сознательно идёт desktop-first и сужает сетку порогами 1100, 768 и 480. HTML подключает один `css/index.css`, который только импортирует файлы. Сборщик не ставят.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет разложить стили по папкам блоков, описать три брейкпоинта курса, спрятать меню в бургер с 768 px, убрать горизонтальный скролл на 375 px и объяснить, почему hover-кнопка карточки на таче должна быть видна всегда.

Как работать с методическими указаниями

- Откройте DevTools и проверяйте 1280, 1024, 768, 375.
- Меняйте сетку блока в его файле, не в общем хвосте медиазапросов.
- Смотрите `document.documentElement.scrollWidth`: он не должен превышать `innerWidth`.
- Прогоните бургер, карточку и форму входа на узкой ширине.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа
2. Структура CSS
3. Брейкпоинты CORAL
4. Поведение блоков
5. Практикумы
6. Типичные ошибки
7. Контрольные вопросы
8. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

Цель занятия — не «ещё три @media в одном файле». Цель — структура, в которой изменение героя не требует поиска правил в конце тысячестрочного листа.

ВАЖНО

@import удобен для учёбы и блокирует отрисовку множеством запросов. В проде файлы склеивают. Здесь важнее читаемая структура.

2. Структура CSS

```
css/
  index.css           ← только @import
  common/variables.css
  common/reset.css
  blocks/header/header.css
  blocks/hero/hero.css
  blocks/product-card/product-card.css
  blocks/modal/modal.css
```

Пример 1 — Дерево этапа 7

3. Брейкпоинты CORAL

Ширина	Сетка	Что меняется
> 1100	4 колонки	Как в макете
≤ 1100	3 колонки	Герой проще, подвал 2×2
≤ 768	2 колонки	Бургер, баннер столбиком
≤ 480	1 колонка	Категории стопкой, модалка шире

Число 768 условно. Важно поведение: меню прячется, сетка меняется. На курс фиксируют три порога и не плодят двенадцать точек «под каждый телефон».

4. Поведение блоков

Горизонтальное меню из семи пунктов не влезает на узкий экран — появляется бургер. Hover на пальце нет: действие карточки нельзя прятать только под :hover. Модалка на 480 px занимает почти всю ширину, но сохраняет крестик и поля с label.

```
.hero { display: grid; grid-template-columns: 1fr 1fr 0.9fr; }
@media (max-width: 768px) {
  .hero { grid-template-columns: 1fr; }
}
```

Пример 2 — Адаптив внутри героя

5. Практикумы

Вынесите любой новый блок в `css/blocks/...` с собственными `@media`.

Проверьте 375 px на горизонтальный скролл.

Откройте бургер и модалку товара.

Сделайте кнопку карточки видимой без `hover`.

6. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Все <code>@media</code> в конце	Блок и его узкая версия живут врозь	Медиа в файле блока
Притворяться <code>mobile-first</code>	Макет <code>desktop</code> , правила путаются	Честный <code>max-width</code>
Семь пунктов меню 10 px	Нечитаемо	Бургер с 768
Действие только <code>:hover</code>	Тач не откроет товар	Видимая кнопка
Сборщик «как в проде»	Скрывает структуру	Импорты браузера

7. Контрольные вопросы

1. Зачем мета `viewport`?
2. Почему курс `desktop-first`?
3. Что меняется на 1100, 768 и 480?
4. Почему медиа пишут в файле блока?
5. Чем `@import` удобен и чем вреден?
6. Как обнаружить горизонтальный скролл?
7. Почему `hover` недостаточно на таче?
8. Зачем бургер, если пункты можно уменьшить?
9. Что подключает HTML: один файл или двадцать?
10. Какое ДЗ закрепляет структуру папок?

8. Итоговая самопроверка

Разберите `header` на трёх ширинах: какие элементы прячутся, какой модификатор открывает меню, где лежит `@media`.

Заключение

Этап 7 делает витрину пригодной к телефону и готовит CSS к росту. Поведение скрипта пока то же. Следующий шаг — разрезать JavaScript на классы.

Источники

Каталог `stage-07-adaptive`.

Лекция `docs/course/lectures/07-adaptive/lecture.md`.

MDN: Responsive design, @media, viewport, @import.

УЧЕБНО-МЕТОДИЧЕСКОЕ ИЗДАНИЕ

ООП · КЛАССЫ · МОДУЛИ

Рефакторинг JavaScript магазина на классах

Методические указания на примере проекта CORAL

Этап 8 — ООП

Назначение	то же поведение, другое устройство кода
Уровень	средний
Исходный проект	Coral_Example / stage-08-oop
Формат работы	ES-модули, наследование модалок, композиция каталога

2026

Аннотация

Методические указания посвящены объектно-ориентированному рефакторингу клиентского кода CORAL. Сценарии этапа 5 сохраняются. Глобальный PRODUCTS и один IIFE заменяют классы Product, Catalog, Modal, AuthModal, ProductModal, StorageBox, App и ES-модули.

Наследование берут только там, где «является»: модалка входа является модалкой. Каталог не наследуют от массива — он содержит карточки. Точка входа — js/App.js с type="module". Нужен локальный сервер.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет объяснить конструктор, extends и super, отличить композицию от наследования, собрать фасад App и показать, что визуальное поведение витрины не изменилось.

Как работать с методическими указаниями

- Запускайте `python -m http.server 8080` из stage-08-oop.
- Сначала прочитайте схему классов на доске, потом файлы.
- Прогоните те же сценарии, что на этапе 5.
- Не тащите Vite: модули уже грузит браузер.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа
2. Класс и объект
3. Наследование и композиция
4. this, модули, фасад
5. Карта классов CORAL
6. Практикумы
7. Типичные ошибки
8. Контрольные вопросы
9. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

Когда появятся корзина и админка, один файл функций перестанет масштабироваться. Классы режут ответственность: данные, DOM, хранилище, фасад. Поведение страницы не меняют — меняют устройство кода.

2. Класс и объект

Класс описывает шаблон, объект — экземпляр. constructor получает узлы и зависимости. Вызов без new даёт неверный this. Статический метод принадлежит классу, как проверки FormValidator. Приватное поле # скрывает деталь. Геттер get price() прячет формат отображения.

3. Наследование и композиция

extends выражает «является». AuthModal и ProductModal разделяют open, close и Escape. В конструкторе наследника вызывают super(). Переопределённый метод может опереться на super.method().

Композиция выражает «имеет». Catalog содержит ProductCard. UserSession и Selection содержат StorageBox. Catalog extends Array не берут: каталог не является массивом, он им пользуется.

4. this, модули, фасад

Обычная функция-слушатель теряет this объекта. На курсе методы привязывают в конструкторе или используют стрелку как поле. export class и import делают зависимости явными. App — фасад: HTML знает одну точку входа, виджеты не лезут в DOMContentLoaded поодиночке.

```
class App {
  constructor() {
    this.catalog = new Catalog("data/products.json");
    this.authModal = new AuthModal(document.getElementById("modal-auth"));
  }
  async start() {
    await this.catalog.load();
    this.catalog.render();
  }
}
```

Пример 1 — Композиция в App

5. Карта классов CORAL

Класс	Роль	Приём
Product	Данные и цена	инкапсуляция
ProductCard	DOM карточки	один объект — кусок UI
Catalog	JSON и сетки	композиция

Modal	open / close / Esc	базовый класс
AuthModal	вход и валидация	наследование
StorageBox	localStorage	повтор
App	сборка и start()	фасад

```

App
├─ Catalog — Product — ProductCard
├─ UserSession — StorageBox
├─ Selection — StorageBox
├─ AuthModal — Modal
└─ ProductModal — Modal

```

Пример 2 — Дерево связей

6. Практикумы

Нарисуйте схему классов по эталонной папке.

Найдите место, где нужен super().

Объясните, почему Catalog не extends Array.

Добавьте учебный класс SearchForm без правки HTML-классов карточки.

7. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Забыли new	this не тот	Всегда new Class()
Наследование «на всякий случай»	Жёсткая связь	Композиция
fetch в конструкторе	Объект не готов, ошибка теряется	async start()
Модалка знает localStorage	Связность растёт	StorageBox отдельно
Глобальный PRODUCTS	Рефакторинг не состоялся	Состояние в экземпляре

8. Контрольные вопросы

1. Чем класс отличается от объекта?
2. Зачем constructor?
3. Когда уместен extends?
4. Что будет, если забыть super()?
5. Чем композиция отличается от наследования?
6. Почему теряется this в DOM-колбэке?
7. Зачем type="module"?
8. Что такое фасад App?

9. Какой принцип нарушает класс, который и рисует, и ходит в сеть, и валидирует пароль?
10. Как понять, что рефакторинг успешен?

9. Итоговая самопроверка

Проведите клик `Select product` по слоям объектов: `ProductCard` сообщает `id`, `ProductModal` запрашивает сессию, `Selection` пишет `StorageBox`, `App` только связал события.

Заключение

Этап 8 готовит клиент к корзине и админке. Данные по-прежнему можно держать в JSON на сервере. Следующий шаг — таблицы SQLite при том же API.

Источники

Каталог `stage-08-oop`.

Лекция `docs/course/lectures/08-oop/lecture.md`.

MDN: `Classes`, `modules`, `this`, `private properties`.

SQLite как хранилище каталога и пользователей

Методические указания на примере проекта CORAL

Этап 9 — База данных

Назначение	заменить JSON-хранилище таблицами при том же API
Уровень	средний
Исходный проект	Coral_Example / stage-09-database
Формат работы	schema.sql, сиды, PDO / ORM / node:sqlite

2026

Аннотация

Методические указания посвящены появлению реляционного хранилища в CORAL. HTTP-контракт этапа 6 не меняют. Вместо store.json появляются таблицы users, products и selected_products в файле coral.db.

Одна schema.sql обслуживает PHP и Node. Django описывает те же сущности моделями и миграциями. Пароль по-прежнему только hash. Админ admin@coral.local закладывается уже здесь, чтобы этап 10 не начинался с пустых users.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет прочитать schema.sql, поднять стек так, чтобы появился coral.db, увидеть восемь товаров из таблицы, зарегистрировать пользователя и объяснить, почему правки products.json после сида не меняют витрину.

Как работать с методическими указаниями

- Сначала читайте схему, потом код доступа к БД.
- После первого запроса проверьте, что файл .db создан.
- Откройте БД в DB Browser или dbshell.
- Никогда не склеивайте SQL со строкой email.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа
2. Таблицы и ключи
3. Запросы и безопасность
4. Три стека
5. Практикумы
6. Типичные ошибки
7. Контрольные вопросы
8. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

JSON-файл плохо отвечает на вопрос «все заказы пользователя X» и плохо переживает одновременную запись. Таблица, первичный ключ и INSERT/SELECT решают это на учебном масштабе без Docker и PostgreSQL.

2. Таблицы и ключи

Таблица	Смысл
users	имя, email UNIQUE, password_hash, is_admin
products	карточки витрины
selected_products	выбор товара из ранних этапов

Первичный ключ однозначно называет строку. Внешний ключ связывает выбор с пользователем и товаром. Сид берёт старый products.json, пока таблица пуста, и создаёт админа.

3. Запросы и безопасность

SELECT читает витрину, INSERT создаёт пользователя, WHERE сужает выборку, JOIN собирает связанные строки, ORDER BY сортирует, COUNT считает. SQLite — один файл, типы гибкие, идентификаторы часто AUTOINCREMENT. Бэкап на этапе учёбы — копия coral.db.

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = ?');
$stmt->execute([$email]);
```

Пример 1 — Плейшолдер вместо склейки

ВАЖНО

Склейка "WHERE email = '\$email'" открывает SQL-инъекцию. Даже учебное поле формы считают враждебным вводом.

4. Три стека

Стек	Файл БД	Как пишут SQL
PHP	php/data/coral.db	PDO + schema.sql
Node 22+	nodejs/data/coral.db	node:sqlite
Django	django/data/coral.db	модели, migrate, seed

После сйда JSON не источник истины. Изменили title в файле и обновили страницу — витрина не изменилась. Это доказательство, что читаете таблицу.

5. Практикумы

Прочитайте schema.sql вслух: три таблицы и связи.

Поднимите PHP и найдите coral.db.

Зарегистрируйтесь и найдите строку в users.

Сделайте Select и найдите selected_products.

Поправьте JSON и убедитесь, что витрина живёт в БД.

6. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Свой набор столбцов у стека	Контракт расходится	Общая schema
Пароль открытым «для отладки»	Утечка	Только hash
Склейка SQL	Инъекция	Плейсхолдер / ORM
Ждать, что JSON всегда источник	Два мира правды	Сид только в пустую таблицу
PostgreSQL в первый день	Операционная сложность	SQLite-файл

7. Контрольные вопросы

1. Почему JSON-файла мало?
2. Что такое первичный и внешний ключ?
3. Зачем UNIQUE на email?
4. Чем SELECT отличается от INSERT?
5. Почему SQLite удобен в курсе?
6. Что такое сид?
7. Как выглядит безопасный запрос в PDO?
8. Зачем Django миграции, если у PHP schema.sql?
9. Почему правка JSON не меняет витрину после сйда?
10. Где уже лежит будущий админ этапа 10?

8. Итоговая самопроверка

Опишите путь GET /api/products: HTTP, выбор стека, SELECT из products, сборка JSON { ok, data }, отсутствие password_hash в ответе.

Заключение

Этап 9 сохраняет API и меняет память приложения. Следующий шаг — корзина, снимок заказа и права администратора.

Источники

[stage-09-database/data/schema.sql](#).

Лекция [docs/course/lectures/09-database/lecture.md](#).

[sqlite.org](#), [PHP PDO](#), [Django models](#), [Node sqlite](#).

Корзина, заказ и админка карточек

Методические указания на примере проекта CORAL

Этап 10 — Покупки и админка

Назначение	превратить «выбор товара» в покупку и управление витриной
Уровень	средний
Исходный проект	Coral_Example / stage-10-shop-admin
Формат работы	роли, корзина, checkout-транзакция, форма новой карточки

2026

Аннотация

Методические указания посвящены финальному слою учебного магазина CORAL. «Выбрать товар» — ещё не покупка. Нужны корзина с количеством, фиксация заказа как снимка цены, история покупок и права: покупатель и администратор.

Новые страницы: `cart.html`, `purchases.html`, `admin.html`. Новые пути: `/api/cart`, `/api/checkout`, `/api/purchases`, `/api/admin/products`. Админ `admin@coral.local` / `admin123`. Ссылку Admin рисуют по полю `isAdmin` из `/api/me`, но защиту даёт сервер, а не CSS.

РЕЗУЛЬТАТ ОБУЧЕНИЯ

После изучения материала студент сможет пройти сценарий покупки, отличить 401 от 403, объяснить снимок цены в `order_items`, добавить карточку из админки и увидеть её на витрине гостя.

Как работать с методическими указаниями

- Запускайте сервер этапа 10, не 6 и не 9.
- Сначала пройдите путь покупателя, затем админа.
- Проверяйте права `curl`-запросом, не только скрытой ссылкой.
- После `checkout` убедитесь, что корзина пуста, а заказ есть в истории.

Оглавление

Аннотация

Как работать с методическими указаниями

1. Назначение этапа

2. Роли и права

3. Корзина и заказ

4. Админка

5. Безопасность ввода

6. Практикумы

7. Типичные ошибки

8. Контрольные вопросы

9. Итоговая самопроверка

Заключение

Источники

1. Назначение этапа

Поверх SQLite появляются `cart_items`, `orders` и `order_items`. Витрина получает Add to cart. Цель курса смыкается: макет, семантика, стили, поведение, сервер, адаптив, классы и база собираются в короткий магазинный сценарий.

2. Роли и права

Аутентификация отвечает на вопрос «кто вы», авторизация — «вам можно». Гость не читает чужую корзину. Одна сессия может быть покупателем или админом. Принцип наименьших привилегий: обычный пользователь не создаёт карточки.

Ситуация	Код
Нет cookie	401
Есть cookie, <code>is_admin=0</code> , POST <code>/api/admin/products</code>	403
Пустая корзина, POST <code>/api/checkout</code>	400

3. Корзина и заказ

Строка корзины живая: количество меняется, ноль удаляет позицию одним POST `/api/cart/update`. Заказ — снимок. Цена копируется в `order_items.price`. Если завтра админ поставит худи 199 вместо 155, вчерашний чек останется 2×155 .

Checkout выполняют транзакцией: либо есть заказ и пустая корзина, либо ничего не изменилось. История `/api/purchases` показывает только свои заказы.

ВАЖНО

Корзина в `localStorage` в обход API снова делает клиент источником истины и ломает права. Количество и сумма считаются на сервере.

4. Админка

Своя страница `admin.html` нужна, чтобы три стека выглядели одинаково для студента. Поля: название, цена больше нуля, URL картинки из `/assets/images/`. Slug и загрузка файла на курсе можно упростить до готового пути. После успешного POST карточка появляется в GET `/api/products`.

5. Безопасность ввода

Название товара из админки не вставляют сырым innerHTML — это XSS. Checkout с cookie уязвим к CSRF в проде; учебный API тему называет, даже если защиту упрощают. Сервер снова проверяет цену, количество и роль. Скрытие ссылки Admin стилем не есть защита.

6. Практикумы

6.1. Покупатель

Зарегистрируйте обычный email.
Положите два товара в корзину.
Измените количество, обнулите одну строку.
Оформите заказ и откройте purchases.html.

6.2. Админ и права

Войдите как admin@coral.local.
Создайте карточку.
Выйдите и найдите её на витрине гостем.
Под обычной сессией вызовите POST /api/admin/products и получите 403.

7. Типичные ошибки

Ошибка	Почему это проблема	Исправление
Цена только из products	История плывёт	Снимок в order_items
Checkout двумя запросами без BEGIN	Заказ без позиций	Транзакция
Защита админки CSS	Любой знает URL	Проверка is_admin
innerHTML названия	XSS	textContent
Сервер этапа 9	Нет новых таблиц	Запуск stage-10

8. Контрольные вопросы

1. Чем корзина отличается от выбранного товара этапа 6?
2. Почему цену копируют в заказ?
3. Что делает quantity = 0?
4. Зачем транзакция на checkout?
5. Чем 401 отличается от 403?
6. Почему ссылка Admin в шапке не равна авторизации?
7. Какие страницы появились на этапе 10?
8. Что такое XSS на названии карточки?
9. Почему гость видит новую карточку после админки?
10. Какой сценарий считать зачётом этапа?

9. Итоговая самопроверка

Расскажите сквозной путь: витрина → корзина → checkout → история и отдельно: админ → новая карточка → витрина гостя. На каждом шаге назовите метод, путь и таблицу.

КРИТЕРИЙ ПОНИМАНИЯ

Студент связывает макет, тег, класс, запрос и строку БД в один пользовательский сценарий, а не перечисляет технологии списком.

Заключение

Этап 10 замыкает курс: из Coral.fig получился магазин с витриной, аккаунтом, корзиной и админкой. Качество по-прежнему измеряется смыслом и договором API, а не числом фреймворков.

Источники

Каталог stage-10-shop-admin.

Лекция docs/course/lectures/10-shop-admin/lecture.md.

Инструкция docs/course/guides/10-shop-admin.md.

MDN: 401/403, XSS, CSRF; SQLite transactions.